PYTHON PROGRAMMING - ASSIGNMENT 3 (CO3)

LISTS, TUPLES, DICTIONARIES AND SETS: SLICING, MUTABILITY, COPYING AND LIST COMPREHENSION

Name:- Balachandru.P

Register Number:- 1912260301C

Slot:- C

Code & Course:- CSA08 - Python Programming

Faculty:- Pokuri Swetha

GitHub Repository Link:- https://github.com/balachandru4562bala-blip/Python-slot-c.git

Contents: Q1 Lists and slicing: daily rainfall records (6) | Q2 Tuples, mutability and copying: city bus-stop map (7) | Q3 Dictionaries: farmer helpline complaint analysis (7) | Q4 Sets: elective registration clash detection (7) | Q5 List comprehension: cleaning sensor readings (6) | Q6 Nested data: a railway timetable (7) | Q7 Design a contact-merging program from first principles (10) | Total 50 marks

All programs were written as .py files and actually run; every output, table and graph in this report comes from a real run on my computer (Python 3.10.12). Timings change a little from run to run, so counts (operations, bytes) are given next to them wherever the counts do not change. Repository name: CSA08-1912260301C-Assignment3, one folder per question (Q1 to Q7), with the source files, the output text files, the results (.json) files and a README.md.

Q1 - LISTS AND SLICING: DAILY RAINFALL RECORDS (6 Marks)

1. Aim

To keep one year of daily rainfall in a list, find the weekly totals with slicing, the wettest and driest week, the maximum and minimum with their positions, a 7-day moving average (naively and with a running sum) and the dry days, and to measure how the cost grows for 365, 36,500 and 365,000 readings.

2. Predictions made before running

- A year has 365 days = 52 x 7 + 1, so I expect 52 full weeks and one leftover day. The weekly totals must add up to the total rainfall.
- The moving average has n - 6 values (359 for n = 365), and an empty list or a list shorter than 7 must give an empty result, not an error.
- Naive additions: each window adds 7 numbers, so 7(n - 6) = 2,513 for n = 365. Running sum: 7 for the first window and 2 operations (one add, one subtract) for each later window, so 7 + 2(n - 7) = 723.
- The time should grow in the same proportion as n (about 1000 times from 365 to 365,000) and the running sum should be the fastest, because it does fewer operations per window.
- append() and pop() at the end should be very cheap; insert(0, x) and pop(0) should be slow because every element has to move.

3. Program

File: Q1/q1_rainfall.py. It generates the readings with the random module (seed 2026, about 60% dry days, the rest from an exponential distribution so that a few days are very wet), and contains the naive moving average (inner loop and slice + sum), the running-sum version, the deque(maxlen=7) version, the dictionary-by-date version, the operation counters and all the measurements.

```python
"""Q1 - Village rain-gauge station: lists and slicing."""
import json
import random
import sys
import time
from collections import deque
from datetime import date, timedelta


def make_readings(n, seed=2026):
    """n daily rainfall readings in mm. About 60% of days are dry (0.0)."""
    rng = random.Random(seed)
    return [0.0 if rng.random() < 0.6 else round(rng.expovariate(1 / 9), 1) for _ in range(n)]


# ---------- weekly totals with slicing ----------
def weekly_totals(readings):
    return [round(sum(readings[i:i + 7]), 1) for i in range(0, len(readings), 7)]


# ---------- moving average: three ways ----------
def ma_naive_loop(r):
    """Naive: add the 7 values of every window one by one (7 additions per window)."""
    out = []
    for i in range(len(r) - 6):
        total = 0
        for j in range(i, i + 7):
            total += r[j]
        out.append(total / 7)
    return out


def ma_naive_slice(r):
    """Naive but with the slice + built-in sum (still 7 additions per window)."""
    return [sum(r[i:i + 7]) / 7 for i in range(len(r) - 6)]


def ma_running(r):
    """Running sum: add the new day, subtract the day that left (2 operations per window)."""
    if len(r) < 7:
        return []
    window = sum(r[:7])
    out = [window / 7]
    for i in range(7, len(r)):
        window += r[i] - r[i - 7]
        out.append(window / 7)
```

```python
    return out


def ma_deque(r):
    """Alternative 1: window kept in collections.deque(maxlen=7)."""
    win = deque(maxlen=7)
    total = 0.0
    out = []
    for x in r:
        if len(win) == 7:
            total -= win[0]
        win.append(x)
        total += x
        if len(win) == 7:
            out.append(total / 7)
    return out


def ma_dict(r, start=date(2026, 1, 1)):
    """Alternative 2: readings stored in a dict keyed by date."""
    d = {start + timedelta(days=i): x for i, x in enumerate(r)}
    out = []
    for i in range(len(r) - 6):
        day = start + timedelta(days=i)
        out.append(sum(d[day + timedelta(days=k)] for k in range(7)) / 7)
    return out


# ---------- counting additions (to compare with my derived formulas) ----------
def count_adds_naive(n):
    adds = 0
    for i in range(n - 6):
        adds += 7          # 7 values added one by one starting from 0
    return adds


def count_ops_running(n):
    ops = 7                # first window
    for i in range(7, n):
        ops += 2           # one addition and one subtraction
    return ops


def best_time(fn, *args, repeat=3):
    best = float("inf")
    for _ in range(repeat):
```

```
        t = time.perf_counter()
        fn(*args)
        best = min(best, time.perf_counter() - t)
    return best


results = {}

# ================ 1. The 365 readings, tasks of the question ================
readings = make_readings(365)
print("First 14 readings (mm):", readings[:14])
print("Number of readings:", len(readings))

weeks = weekly_totals(readings)
full = weeks[:52]
print("\nWeekly totals: %d full weeks + 1 extra day (365 = 52*7 + 1)" % len(full))
print("First 8 weekly totals:", weeks[:8])
print("Sum of the weekly totals = %.1f ; sum of all readings = %.1f" % (sum(weeks), sum(readings)))
wet = max(range(52), key=lambda w: full[w])
dry = min(range(52), key=lambda w: full[w])
print("Wettest full week: week %d (days %d-%d) with %.1f mm" % (wet + 1, wet * 7 + 1, wet * 7 + 7, full[wet]))
print("Driest full week : week %d (days %d-%d) with %.1f mm" % (dry + 1, dry * 7 + 1, dry * 7 + 7, full[dry]))

mx, mn = max(readings), min(readings)
print("\nMaximum reading %.1f mm on day %d (index %d)" % (mx, readings.index(mx) + 1, readings.index(mx)))
all_min = [i for i, x in enumerate(readings) if x == mn]
print("Minimum reading %.1f mm; first on day %d, it happens on %d days" % (mn, all_min[0] + 1, len(all_min)))

ma = ma_running(readings)
print("\n7-day moving average: %d values (365 - 6)" % len(ma))
print("First 5:", [round(x, 2) for x in ma[:5]])
print("Naive result equals running-sum result:",
      all(abs(a - b) < 1e-9 for a, b in zip(ma_naive_loop(readings), ma)))

dry_days = [i + 1 for i, x in enumerate(readings) if x == 0]
print("\nDry days (0 mm): %d days. First 15 day numbers: %s" % (len(dry_days), dry_days[:15]))

print("\nSlicing: what readings[7:14], readings[-7:] and readings[::7] return")
print("readings[7:14]  ->", readings[7:14], "(%d values, days 8-14)" % len(readings[7:14]))
print("readings[-7:]   ->", readings[-7:], "(last 7 days)")
print("readings[::7]   ->", len(readings[::7]), "values, first 6:", readings[::7][:6], "(days 1, 8, 15, ...)")

# ================ 2. Edge cases ================
print("\nEdge cases")
for label, data in [("empty list", []), ("3 readings", [1.0, 2.0, 3.0]), ("exactly 7", [1.0] * 7), ("8 readings", [1.0] * 7 + [8.0])]:
    outs = [fn(data) for fn in (ma_naive_loop, ma_naive_slice, ma_running, ma_deque)]
    same = all(o == outs[0] for o in outs)
    print("  %-11s -> %s   (all versions same: %s)" % (label, outs[0], same))
print("  weekly_totals([]) ->", weekly_totals([]), "| max of empty list raises:", end=" ")
try:
    max([])
except ValueError as e:
    print("ValueError:", e)

# ================ 3. Derived counts against counted counts ================
print("\nAdditions: derived formula vs counted")
print("%8s %14s %14s %14s %14s" % ("n", "naive 7(n-6)", "naive counted", "running 7+2(n-7)", "running counted"))
count_rows = []
for n in (365, 36500, 365000):
    a, b = 7 * (n - 6), 7 + 2 * (n - 7)
    count_rows.append([n, a, count_adds_naive(n), b, count_ops_running(n)])
    print("%8d %14d %14d %14d %14d" % tuple(count_rows[-1]))
results["counts"] = count_rows

# ================ 4. Timings ================
print("\nTimings in seconds (best of 3). Same random data for every version.")
print("%8s %12s %12s %12s %12s %12s" % ("n", "naive loop", "naive slice", "running", "deque", "dict by date"))
time_rows = []
for n in (365, 36500, 365000):
    r = make_readings(n)
    t = [best_time(ma_naive_loop, r), best_time(ma_naive_slice, r), best_time(ma_running, r),
         best_time(ma_deque, r), best_time(ma_dict, r)]
    ok = all(abs(x - y) < 1e-6 for x, y in zip(ma_running(r), ma_naive_slice(r)))
    time_rows.append([n] + t)
    print("%8d %12.6f %12.6f %12.6f %12.6f %12.6f   same answers: %s" % ((n,) + tuple(t) + (ok,)))
results["times"] = time_rows

# ================ 5. Memory ================
print("\nMemory with sys.getsizeof (the list object only, and with the float objects)")
mem_rows = []
for n in (365, 36500, 365000):
    r = make_readings(n)
    lst = sys.getsizeof(r)
    flo = sum(sys.getsizeof(x) for x in r)
    ds = sys.getsizeof({i: x for i, x in enumerate(r)})
    dq = sys.getsizeof(deque(r[:7], maxlen=7))
    mem_rows.append([n, lst, lst + flo, dq, ds])
    print("  n=%7d list=%9d B  list+floats=%9d B  deque(7)=%d B  dict of n entries=%9d B" % (n, lst, lst + flo, dq, ds))
results["memory"] = mem_rows

# ================ 6. Cost of list operations (n = 365,000) ================
print("\nCost of list operations on a list of 365,000 readings (average of 200 calls, microseconds)")
```

```
base = make_readings(365000)
op_rows = []
for name, fn in [
    ("append(x)", lambda L: L.append(1.0)),
    ("pop()", lambda L: L.pop()),
    ("insert(0, x)", lambda L: L.insert(0, 1.0)),
    ("insert(middle, x)", lambda L: L.insert(len(L) // 2, 1.0)),
    ("pop(0)", lambda L: L.pop(0)),
]:
    L = list(base)
    t = time.perf_counter()
    for _ in range(200):
        fn(L)
    us = (time.perf_counter() - t) / 200 * 1e6
    op_rows.append([name, us])
    print("  %-18s %10.2f us" % (name, us))
L = list(base)
t = time.perf_counter()
L.sort()
sort_s = time.perf_counter() - t
print("  sort() of 365,000   %10.4f s (random order)" % sort_s)
t = time.perf_counter()
L.sort()
print("  sort() again        %10.4f s (already sorted)" % (time.perf_counter() - t))
results["ops"] = op_rows

# ================ 7. Late reading ================
print("\nLate reading for day 100 of a 365-day list")
L = list(readings)
t = time.perf_counter()
for _ in range(1):
    L.insert(99, 3.2)
print("  insert(99, 3.2): length becomes", len(L), "-> every later day moved one place right (day 101 is now at index", 100, ")")
fixed = [None] * 365           # alternative design: one slot per day, late reading goes in its own slot
fixed[99] = 3.2
print("  slot per day: fixed[99] = 3.2 changes only one cell; the length stays", len(fixed))

with open("results_q1.json", "w") as f:
    json.dump(results, f)
```

4. Output

Real run of q1_rainfall.py (command: python3 q1_rainfall.py), Python 3.10.12.

Terminal - python3

```
$ python3 q1_rainfall.py
First 14 readings (mm): [0.0, 0.0, 0.0, 1.0, 0.0, 7.3, 7.1, 13.2, 7.9, 0.0, 1.1, 5.4, 10.4, 2.1]
Number of readings: 365

Weekly totals: 52 full weeks + 1 extra day (365 = 52*7 + 1)
First 8 weekly totals: [15.4, 40.1, 95.6, 15.1, 8.3, 8.6, 34.2, 48.8]
Sum of the weekly totals = 1458.9 ; sum of all readings = 1458.9
Wettest full week: week 3 (days 15-21) with 95.6 mm
Driest full week : week 33 (days 225-231) with 0.0 mm

Maximum reading 71.8 mm on day 17 (index 16)
Minimum reading 0.0 mm; first on day 1, it happens on 214 days

7-day moving average: 359 values (365 - 6)
First 5: [2.2, 4.09, 5.21, 5.21, 5.23]
Naive result equals running-sum result: True

Dry days (0 mm): 214 days. First 15 day numbers: [1, 2, 3, 5, 10, 15, 18, 19, 20, 21, 22, 23, 25, 26, 28]

Slicing: what readings[7:14], readings[-7:] and readings[::7] return
readings[7:14]  -> [13.2, 7.9, 0.0, 1.1, 5.4, 10.4, 2.1] (7 values, days 8-14)
readings[-7:]   -> [0.0, 0.0, 0.7, 7.0, 0.0, 0.0, 0.0] (last 7 days)
readings[::7]   -> 53 values, first 6: [0.0, 13.2, 0.0, 0.0, 0.0, 2.7] (days 1, 8, 15, ...)

Edge cases
  empty list  -> []   (all versions same: True)
  3 readings  -> []   (all versions same: True)
  exactly 7   -> [1.0]   (all versions same: True)
  8 readings  -> [1.0, 2.0]   (all versions same: True)
  weekly_totals([]) -> [] | max of empty list raises: ValueError: max() arg is an empty sequence

Additions: derived formula vs counted
       n   naive 7(n-6)  naive counted running 7+2(n-7) running counted
     365           2513           2513            723             723
   36500         255458         255458          72993           72993
```

```
Terminal - python3
  ... (output continues)
  365000        2554958        2554958         729993         729993

Timings in seconds (best of 3). Same random data for every version.
       n   naive loop  naive slice      running        deque dict by date
     365     0.000097     0.000050     0.000027     0.000046     0.000986   same answers: True
   36500     0.010864     0.005353     0.002914     0.004598     0.105313   same answers: True
  365000     0.111269     0.055619     0.030634     0.047238     1.101546   same answers: True

Memory with sys.getsizeof (the list object only, and with the float objects)
  n=    365 list=     3256 B  list+floats=    12016 B  deque(7)=624 B  dict of n entries=    18520 B
  n=  36500 list=   312024 B  list+floats=  1188024 B  deque(7)=624 B  dict of n entries=  1310808 B
  n= 365000 list=  2926776 B  list+floats= 11686776 B  deque(7)=624 B  dict of n entries= 20971608 B

Cost of list operations on a list of 365,000 readings (average of 200 calls, microseconds)
  append(x)               0.09 us
  pop()                   0.09 us
  insert(0, x)          122.53 us
  insert(middle, x)      40.96 us
  pop(0)                777.87 us
  sort() of 365,000      0.0206 s (random order)
  sort() again           0.0015 s (already sorted)

Late reading for day 100 of a 365-day list
  insert(99, 3.2): length becomes 366 -> every later day moved one place right (day 101 is now at index 100 )
  slot per day: fixed[99] = 3.2 changes only one cell; the length stays 365
```

5. Analysis

- **Weekly totals.** The first 14 readings and the weekly totals come from slicing: weeks = [readings[i:i+7] for i in range(0, 365, 7)]. There are 52 full weeks and a last slice with 1 day, so I searched the wettest and driest week only among the 52 full weeks. Week 3 (days 15-21) was the wettest with 95.6 mm and week 33 (days 225-231) the driest with 0.0 mm. The weekly totals add up to 1458.9 mm, the same as the sum of all readings, so no day was lost or counted twice.
- **Maximum, minimum and dry days.** The maximum is 71.8 mm on day 17 (index 16). The minimum is 0.0 mm and it happens on 214 days, so max() and min() give the value but index() only gives the first position (day 1). To get all positions I used a list comprehension with enumerate. Dry days are 214 of 365 (59%), which is close to the 60% I put in the generator.
- **The three slices.** readings[7:14] returns 7 values, from index 7 to 13, which are days 8 to 14 (the end index is not included). readings[-7:] returns the last 7 days. readings[::7] takes every 7th reading starting from index 0, which is days 1, 8, 15, ... and gives 53 values for a year of 365 days. All three make a new list (a copy of those references), they do not change readings.
- **Why a list is suitable.** A list keeps the order of the days, gives any day by its index in O(1), can be sliced, and can grow. append() and pop() at the end are O(1) on average. insert(i, x) and pop(0) are O(n) because the elements after position i must move. sort() is O(n log n). My measurement on a list of 365,000 readings: append 0.09 us, pop() 0.09 us, insert at the front 122.5 us, insert in the middle 41.0 us, pop(0) 777.9 us. Sorting 365,000 random readings took 0.0206 s, but sorting the already sorted list again took 0.0015 s because Python's sort notices the existing order. insert(0) was cheaper than pop(0) in this run although both are O(n); I think this is just how the memory is moved, so I only trust the O(n) conclusion and not that small difference.

- **Derivation of the additions.** For the naive method each window of 7 values needs 7 additions (counting the first one with the starting 0), and there are n - 6 windows, so the total is 7(n - 6), which is O(7n) = O(n). With a running sum the first window costs 7 additions, and after that each new window needs one addition (new day) and one subtraction (day that left), so 7 + 2(n - 7). My counters gave exactly the same numbers as the formulas (Table Q1-1).

n	naive 7(n-6)	naive counted	running 7+2(n-7)	running counted
365	2,513	2,513	723	723
36,500	255,458	255,458	72,993	72,993
365,000	2,554,958	2,554,958	729,993	729,993

Table Q1-1: additions by formula and by counting in the program (they are equal).

n	naive (inner loop)	naive (slice + sum)	running sum	deque(maxlen=7)	dict by date
365	0.00010 s	0.00005 s	0.00003 s	0.00005 s	0.00099 s
36,500	0.01086 s	0.00535 s	0.00291 s	0.00460 s	0.10531 s
365,000	0.11127 s	0.05562 s	0.03063 s	0.04724 s	1.10155 s

Table Q1-2: moving-average time, best of 3 runs (all versions gave the same answers).

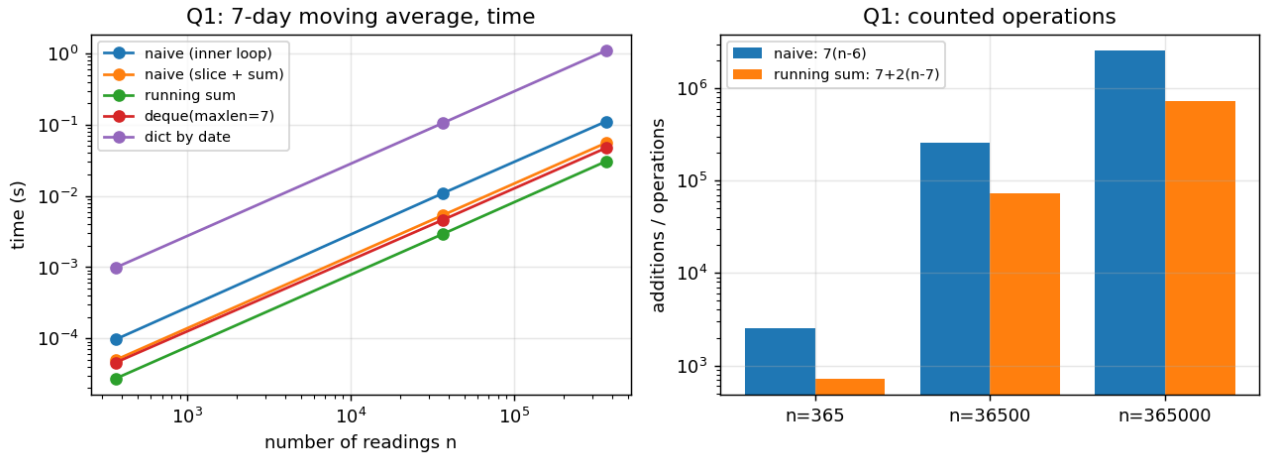


Figure Q1-1: time (log scale) and number of operations for the moving average.

The predictions held. The time grew about 1000 times when n grew 1000 times (naive inner loop: 0.000097 s to 0.111 s), so it is linear, O(n). The running sum was the fastest: 3.6 times faster than the naive inner loop and 1.8 times faster than the naive slice + sum. The counted operations differ by 3.5 times (2,554,958 against 729,993 for n = 365,000), and the time ratio is in the same range, so the counts explain the times. The slice + sum version is quicker than the inner loop although both do 7n additions, because sum() runs in C and the inner loop runs in Python.

Edge cases: an empty list and a list of 3 readings give [] from every version (no window of 7 exists); exactly 7 readings give 1 value and 8 readings give 2. The only error I got was max([]), which raises ValueError, so the program must check for an empty list before asking for the maximum of the readings.

6. Design Question: a late reading

list.insert(i, x) puts the value at position i and moves every element after i one place to the right, so it costs O(n - i). On my list of 365,000 readings insert in the middle took 41.0 us and at the front 122.5 us, which is nothing for one reading but it adds up if many readings arrive late. There is a second, bigger problem: after

the insert, day 101 is at index 100 and every later day is shifted, so the rule "day number = index + 1" breaks, and all the weekly slices after that point cut at the wrong place. For a year of 365 values I would still use a list, but with one slot for every day (None when the reading has not arrived), so a late reading is just fixed[99] = 3.2: O(1), the length stays 365 and nothing moves (the last lines of the output show this). If the readings were sparse over many years I would use a dictionary keyed by date.

7. Alternative Concept and Complexity Analysis

With collections.deque(maxlen=7) the window itself is kept in the deque: append the new reading and the oldest one drops out automatically, and a running total is updated. With a dictionary keyed by date each window is built from 7 lookups. Both give the same 359 averages as the list version (checked for all three sizes).

n	list (B)	list + float objects (B)	deque of 7 (B)	dict of n entries (B)
365	3,256	12,016	624	18,520
36,500	312,024	1,188,024	624	1,310,808
365,000	2,926,776	11,686,776	624	20,971,608

Table Q1-3: memory with sys.getsizeof (the dict figure does not include the float objects or the date keys).

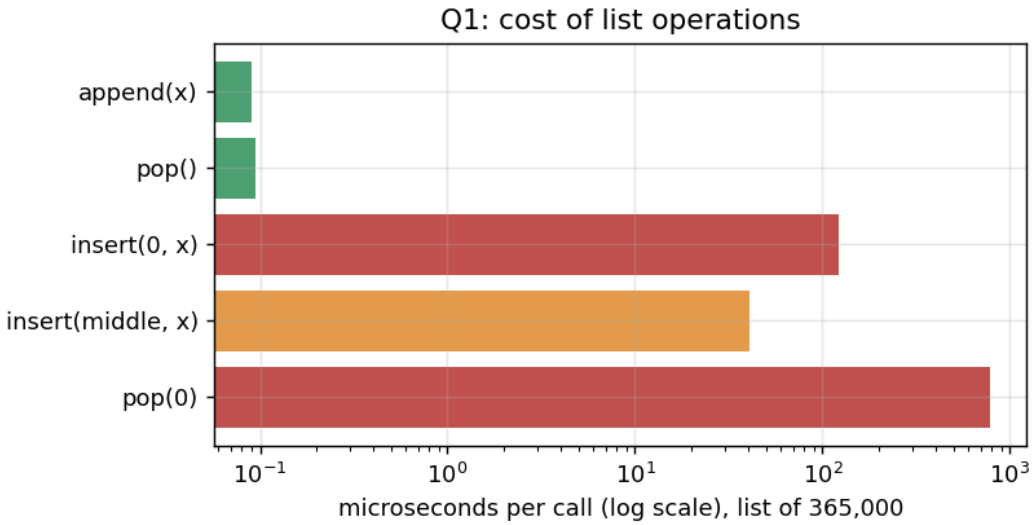


Figure Q1-2: cost of list operations on 365,000 readings (log scale).

Time: slicing (naive) is O(7n) = O(n); the running sum, the deque and the dictionary version are all O(n) in the number of windows, but the dictionary version does 7 lookups and 7 date additions per window, so it is O(7n) with a big constant: 1.10155 s against 0.03063 s for the running sum at n = 365,000 (about 36 times slower). Space: the list of readings is O(n) in all cases; the deque needs only O(7) extra (624 bytes), the slices make a temporary 7-value list, and the dictionary is O(n) with a larger constant (20,971,608 B for the dict against 2,926,776 B for the list at n = 365,000). So the best design measured here is the running sum on a list; the deque is the best when the readings arrive one by one (streaming) and the whole year need not be stored.

8. Conclusion

Slicing made the weekly totals and the moving average short to write, and a list was a good fit for one year of readings. The most useful result was the counting of operations: 7(n - 6) against 7 + 2(n - 7) matched my counters exactly and explained the measured speed-up of the running sum. I also learned that the nice formula is not the whole story, because sum() in C beat my hand-written inner loop even though both do the same number of additions.

Q2 - TUPLES, MUTABILITY AND COPYING: CITY BUS-STOP MAP (7 Marks)

1. Aim

To store bus stops as (latitude, longitude) tuples and use them as dictionary keys, to build the zone grid correctly, to show aliasing, shallow copies and deep copies with is and id(), to explain why a function can change the caller's list, and to measure a tuple against a list.

2. Predictions made before running

- [[0] * 4] * 3 repeats a reference to one row, so bad[0] is bad[1] will be True and one update will show in all rows; the comprehension version will give False and change one row.
- A list as a dictionary key will fail with TypeError: unhashable type: 'list'.
- copy.copy and list[:] will copy only the outer list, so an inner list is still shared; copy.deepcopy will copy everything.
- A tuple of 1,000,000 numbers should take a little less memory than a list (no spare room), but not much less, because both store 8-byte references. Indexing should take the same time.
- A tuple cannot be changed, but if it holds a list, that list can still be changed.

3. Program: Complete the Code (TODO 1)

File: Q2/q2_complete_code.py. The given program is unchanged; only TODO 1 is replaced with good = [[0] * cols for _ in range(rows)]. I also added the last lines that try a list as a dictionary key.

```python
"""Q2 - Complete the Code (TODO 1 completed, the rest of the given program unchanged)."""
rows, cols = 3, 4

bad = [[0] * cols] * rows   # every row is the SAME list object
bad[0][1] = 7

# TODO 1: Create a rows x cols grid of zeros using a list comprehension so that every
# row is a separate list
good = [[0] * cols for _ in range(rows)]   # my code: a new inner list is built for every row

good[0][1] = 7

print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])

# Tuples are immutable, so they can be used as dictionary keys
stops = {(13.0827, 80.2707): "Chennai Central", (13.0418, 80.2341): "T. Nagar"}

print(stops[(13.0418, 80.2341)])

# Try to use a list [13.0418, 80.2341] as a dictionary key and explain the error.
try:
    stops[[13.0418, 80.2341]] = "T. Nagar again"
except TypeError as err:
    print("TypeError:", err)
```

4. Output of the Complete the Code program

Real run (command: python3 q2_complete_code.py), Python 3.10.12.



The first four lines are exactly the expected output of the question: bad shows 7 in the second place of every row, good only in the first row, "True False" for the two is tests, and T. Nagar. The extra line is the answer to the last instruction: stops[[13.0418, 80.2341]] raises TypeError: unhashable type: 'list'. A dictionary must compute a hash of the key and keep it for ever; a list can be changed, so its content (and so its hash) could change after it is stored and the dictionary would never find the key again. A tuple of numbers cannot change, so its hash is stable. A tuple that contains a list is also unhashable (shown in the investigation run), because the hash needs the hash of every item.

5. Program: the investigation

File: Q2/q2_tuples.py. It unpacks the tuple keys in a for loop, shows the bug and the fix with id(), compares aliasing, copy.copy, slicing and deepcopy, shows how a function changes its list argument, tests a list inside a tuple, measures memory and time for 1,000,000 values, and measures the dictionary grid that is used in the alternative concept.

```python
"""Q2 - Tuples, mutability and copying: bus-stop map investigation."""
import copy
import json
import sys
import time

results = {}

# ---------- 1. tuples as keys, unpacking in a for loop ----------
stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar",
    (13.0674, 80.2376): "Egmore",
    (12.9249, 80.1000): "Tambaram",
}
print("Unpacking tuple keys in a for loop:")
for (lat, lon), name in stops.items():
    print("  %-16s lat=%.4f lon=%.4f" % (name, lat, lon))
print("hash of a tuple works:", hash((13.0418, 80.2341)) != 0)
try:
    hash([13.0418, 80.2341])
except TypeError as e:
    print("hash of a list fails:", e)
try:
    hash((13.0418, [80.2341]))
except TypeError as e:
    print("hash of a tuple that CONTAINS a list fails too:", e)

# ---------- 2. the bug and the fix, with is and id() ----------
rows, cols = 3, 3
bad = [[0] * cols] * rows
good = [[0] * cols for _ in range(rows)]
print("\nid() of each row")
print("  bad  rows:", [id(r) for r in bad])
print("  good rows:", [id(r) for r in good])
print("  bad[0] is bad[1]:", bad[0] is bad[1], "| good[0] is good[1]:", good[0] is good[1])
bad[1][2] = 5
good[1][2] = 5
print("  after grid[1][2] = 5 ->")
print("  bad :", bad)
print("  good:", good)
print("  the number 0 inside the rows is shared in BOTH grids:", good[0][0] is good[1][0])

# ---------- 3. aliasing, shallow copy, deep copy ----------
orig = [[1, 2], [3, 4]]
alias = orig
```

```
shallow1 = copy.copy(orig)
shallow2 = orig[:]
deep = copy.deepcopy(orig)
print("\nCopies of", orig)
print("  alias is orig:", alias is orig, "| copy.copy is orig:", shallow1 is orig,
      "| slice is orig:", shallow2 is orig, "| deepcopy is orig:", deep is orig)
print("  inner rows shared?  alias:", alias[0] is orig[0], " copy.copy:", shallow1[0] is orig[0],
      " slice:", shallow2[0] is orig[0], " deepcopy:", deep[0] is orig[0])
orig.append([5, 6])      # change the OUTER list
orig[0][0] = 99          # change an INNER list
print("  after orig.append([5, 6]) and orig[0][0] = 99:")
print("  orig   :", orig)
print("  alias  :", alias)
print("  copy   :", shallow1)
print("  slice  :", shallow2)
print("  deep   :", deep)

# ---------- 4. function that changes its list argument ----------
def add_stop(route, stop):
    route.append(stop)          # changes the caller's list

def add_stop_safe(route, stop):
    return route + [stop]       # builds a new list

route = ["Central", "Egmore"]
print("\nFunction argument")
print("  id(route) before:", id(route))
add_stop(route, "Tambaram")
print("  after add_stop(route, 'Tambaram'):", route, id(route))
new = add_stop_safe(route, "Guindy")
print("  after add_stop_safe:", route, "| returned:", new, "| different object:", new is not route)

# ---------- 5. list inside a tuple ----------
t = ("Route 5", [10, 20])
t_id = id(t)
print("\nList stored inside a tuple:", t, "id of tuple", id(t), "id of inner list", id(t[1]))
t[1].append(30)
print("  t[1].append(30) works  ->", t, "(tuple still the same object:", id(t) == t_id, ")")
try:
    t[0] = "Route 6"
except TypeError as e:
    print("  t[0] = 'Route 6' fails:", e)
try:
    t[1] += [40]
except TypeError as e:
    print("  t[1] += [40] raises:", e)
```

```
print("  ... but the list was still changed:", t)

# ---------- 6. memory of 1,000,000 values ----------
N = 1_000_000
L = list(range(N))
T = tuple(range(N))
print("\nsys.getsizeof for 1,000,000 values")
print("  list :", sys.getsizeof(L), "bytes")
print("  tuple:", sys.getsizeof(T), "bytes")
print("  difference:", sys.getsizeof(L) - sys.getsizeof(T), "bytes")
print("  for a short one: list of 3 =", sys.getsizeof([1, 2, 3]), "bytes, tuple of 3 =", sys.getsizeof((1, 2, 3)), "bytes")
results["mem"] = [sys.getsizeof(L), sys.getsizeof(T)]

# ---------- 7. time: create and index ----------
def best(fn, repeat=5):
    b = 1e9
    for _ in range(repeat):
        t0 = time.perf_counter(); fn(); b = min(b, time.perf_counter() - t0)
    return b

src = list(range(N))
create_list = best(lambda: list(src))
create_tuple = best(lambda: tuple(src))
create_lit_list = best(lambda: [1, 2, 3, 4, 5] * 1)
def index_all(seq):
    s = 0
    for i in range(0, N, 1):
        s += seq[i]
    return s
idx_list = best(lambda: index_all(L), 3)
idx_tuple = best(lambda: index_all(T), 3)
print("\nTime (best of runs), 1,000,000 values")
print("  create from a list : list %.5f s | tuple %.5f s" % (create_list, create_tuple))
print("  index every item   : list %.5f s | tuple %.5f s" % (idx_list, idx_tuple))
import timeit
lit_l = timeit.timeit("[1, 2, 3, 4, 5]", number=2_000_000)
lit_t = timeit.timeit("(1, 2, 3, 4, 5)", number=2_000_000)
print("  literal of 5 numbers x 2,000,000: list %.4f s | tuple %.4f s" % (lit_l, lit_t))
results["time"] = [create_list, create_tuple, idx_list, idx_tuple, lit_l, lit_t]

# ---------- 8. alternative: dict grid keyed by (row, col) ----------
print("\nAlternative: dict keyed by (row, col), only non-empty zones")
import random
rng = random.Random(7)
alt_rows = []
for R, C, frac in [(100, 100, 0.05), (1000, 1000, 0.01), (2000, 2000, 0.005)]:
```

```
    t0 = time.perf_counter()
    grid = [[0] * C for _ in range(R)]
    build_l = time.perf_counter() - t0
    cells = [(rng.randrange(R), rng.randrange(C)) for _ in range(int(R * C * frac))]
    t0 = time.perf_counter()
    for r_, c_ in cells:
        grid[r_][c_] += 1
    upd_l = time.perf_counter() - t0
    sparse = {}
    t0 = time.perf_counter()
    for key in cells:
        sparse[key] = sparse.get(key, 0) + 1
    upd_d = time.perf_counter() - t0
    mem_l = sys.getsizeof(grid) + sum(sys.getsizeof(row) for row in grid)
    mem_d = sys.getsizeof(sparse) + sum(sys.getsizeof(k) for k in sparse)   # dict + its tuple keys
    same = all(grid[r_][c_] == v for (r_, c_), v in sparse.items())
    alt_rows.append([R, C, len(sparse), mem_l, mem_d, upd_l, upd_d])
    print("  %dx%d, %d non-empty zones: list-of-lists %9d B, dict %8d B | updates %d: list %.4f s, dict %.4f s | same counts: %s"
          % (R, C, len(sparse), mem_l, mem_d, len(cells), upd_l, upd_d, same))
results["alt"] = alt_rows

with open("results_q2.json", "w") as f:
    json.dump(results, f)
```

6. Output of the investigation

Real run (command: python3 q2_tuples.py), Python 3.10.12. The id numbers change from run to run, only the equal/not equal pattern matters.

```
Terminal - python3

$ python3 q2_tuples.py
Unpacking tuple keys in a for loop:
  Chennai Central  lat=13.0827 lon=80.2707
  T. Nagar         lat=13.0418 lon=80.2341
  Egmore           lat=13.0674 lon=80.2376
  Tambaram         lat=12.9249 lon=80.1000
hash of a tuple works: True
hash of a list fails: unhashable type: 'list'
hash of a tuple that CONTAINS a list fails too: unhashable type: 'list'

id() of each row
  bad  rows: [139621694484160, 139621694484160, 139621694484160]
  good rows: [139621696109824, 139621694824512, 139621694829120]
  bad[0] is bad[1]: True | good[0] is good[1]: False
  after grid[1][2] = 5 ->
  bad : [[0, 0, 5], [0, 0, 5], [0, 0, 5]]
  good: [[0, 0, 0], [0, 0, 5], [0, 0, 0]]
  the number 0 inside the rows is shared in BOTH grids: True

Copies of [[1, 2], [3, 4]]
  alias is orig: True | copy.copy is orig: False | slice is orig: False | deepcopy is orig: False
  inner rows shared?  alias: True  copy.copy: True  slice: True  deepcopy: False
  after orig.append([5, 6]) and orig[0][0] = 99:
  orig   : [[99, 2], [3, 4], [5, 6]]
  alias  : [[99, 2], [3, 4], [5, 6]]
  copy   : [[99, 2], [3, 4]]
  slice  : [[99, 2], [3, 4]]
  deep   : [[1, 2], [3, 4]]

Function argument
  id(route) before: 139621694824896
  after add_stop(route, 'Tambaram'): ['Central', 'Egmore', 'Tambaram'] 139621694824896
  after add_stop_safe: ['Central', 'Egmore', 'Tambaram'] | returned: ['Central', 'Egmore', 'Tambaram', 'Guindy']
     | different object: True
```

```
Terminal - python3

  ... (output continues)
List stored inside a tuple: ('Route 5', [10, 20]) id of tuple 139621696189952 id of inner list 139621694824768
  t[1].append(30) works  -> ('Route 5', [10, 20, 30]) (tuple still the same object: True )
  t[0] = 'Route 6' fails: 'tuple' object does not support item assignment
  t[1] += [40] raises: 'tuple' object does not support item assignment
  ... but the list was still changed: ('Route 5', [10, 20, 30, 40])

sys.getsizeof for 1,000,000 values
  list : 8000056 bytes
  tuple: 8000040 bytes
  difference: 16 bytes
  for a short one: list of 3 = 88 bytes, tuple of 3 = 64 bytes

Time (best of runs), 1,000,000 values
  create from a list : list 0.00342 s | tuple 0.00346 s
  index every item   : list 0.03035 s | tuple 0.03018 s
  literal of 5 numbers x 2,000,000: list 0.0485 s | tuple 0.0104 s

Alternative: dict keyed by (row, col), only non-empty zones
  100x100, 493 non-empty zones: list-of-lists     86520 B, dict    46128 B | updates 500: list 0.0000 s, dict
    0.0000 s | same counts: True
  1000x1000, 9941 non-empty zones: list-of-lists   8064856 B, dict   851696 B | updates 10000: list 0.0014 s,
    dict 0.0011 s | same counts: True
  2000x2000, 19944 non-empty zones: list-of-lists  32128184 B, dict  1706784 B | updates 20000: list 0.0024 s,
    dict 0.0019 s | same counts: True
```

7. Analysis

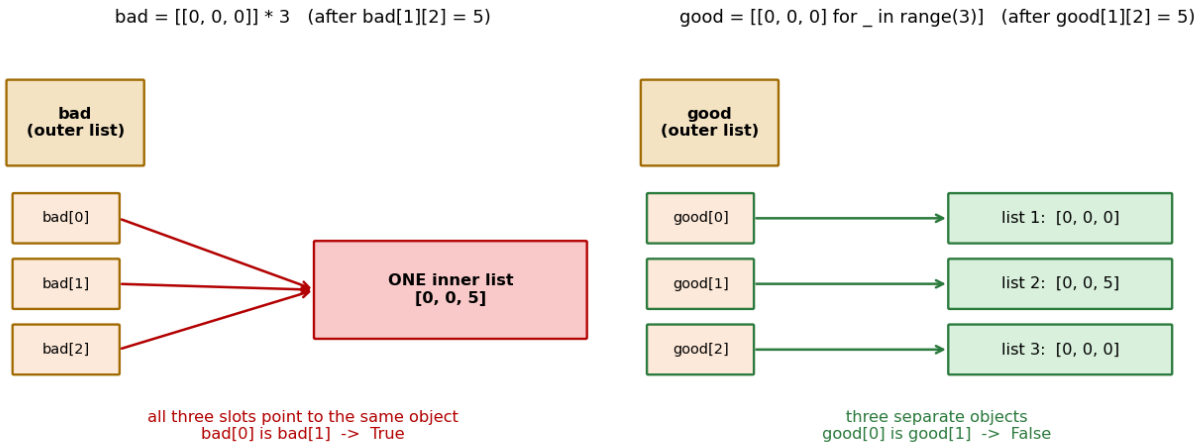


*Figure Q2-1: memory diagram of [[0] * 3] * 3 (left) and of the corrected grid (right). The outer list holds references; in the bad grid all three references go to one inner list.*

- **The bug and the correction.** [[0] * cols] * rows builds the inner list once and then repeats the reference rows times. The id() of the three rows is the same number, bad[0] is bad[1] is True, and bad[1][2] = 5 appears in every row. The comprehension runs [0] * cols again for every row, so the three rows have three different ids and only good[1] changes. The number 0 itself is shared in both grids (good[0][0] is good[1][0] is True), but that is harmless because integers cannot be changed.
- **Aliasing, shallow and deep copy.** Aliasing (alias = orig) makes a second name for the same list: alias is orig is True and every change shows in both. copy.copy(orig) and orig[:] make a new outer list, but the inner rows are the same objects (shared = True for both). So after orig.append([5, 6]) the copies did not get the new row, but after orig[0][0] = 99 both copies show 99 because they share the inner row. copy.deepcopy copied the inner lists too, so it stayed [[1, 2], [3, 4]].
- **When to use which.** A shallow copy is enough when the items are immutable (numbers, strings, tuples) or when I only change the outer list (append, remove, sort). A deep copy is needed when the list contains other mutable objects (a grid, a list of lists, a list of dictionaries) and I want to change them independently, for example to try a new bus route plan without changing the live one. The price of deepcopy is more time and memory.
- **Function that changes the caller's list.** The parameter route is just another name for the same list object (same id before and after). add_stop does route.append(...), which changes that shared object, so the caller's list changes. add_stop_safe builds a new list with route + [stop], and the caller's list stayed the same. If a function should not change its argument, it should copy or return a new list.
- **A list inside a tuple.** t = ("Route 5", [10, 20]): the tuple cannot change which objects it holds, but the list inside is a normal list, so t[1].append(30) worked and the tuple now shows [10, 20, 30]. t[0] = "Route 6" failed with TypeError. The interesting case is t[1] += [40]: Python first extends the list (this works), then tries to store the result back into t[1] and that raises TypeError, so I got the error AND the list was changed anyway. The tuple is immutable only in its references, not in the objects it points to ("shallow immutability").

Measurement (1,000,000 values)	list	tuple
sys.getsizeof (bytes)	8,000,056	8,000,040
create from an existing list	0.00342 s	0.00346 s
index every item with a for loop	0.03035 s	0.03018 s
literal of 5 numbers x 2,000,000	0.0485 s	0.0104 s
getsizeof of 3 values	88	64

Table Q2-1: list against tuple.

The memory prediction was right: the tuple is only 16 bytes smaller than the list for 1,000,000 values (8,000,056 against 8,000,040), because the list has the same 8 bytes per reference, and indexing took the same time. For a tiny object the difference is clearer (88 against 64 bytes for 3 values). A new thing I found: a literal like (1, 2, 3, 4, 5) is made once when Python compiles the code, so it was about 5 times faster than building the list [1, 2, 3, 4, 5] every time.

8. Alternative Concept and Complexity Analysis

Instead of a list of lists, the zone grid can be a dictionary keyed by (row, col) tuples that holds only the zones with passengers: grid[(r, c)] = grid.get((r, c), 0) + 1, and a zone that is not in the dictionary has 0. This works because the tuple (row, col) is hashable. I tested it for three grid sizes with about 0.5% to 5% of the zones used, and the counts were the same as in the list grid.

Grid	zones used	list of lists (B)	dict + tuple keys (B)	update time list	update time dict
100x100	493	86,520	46,128	0.0000 s	0.0000 s
1000x1000	9,941	8,064,856	851,696	0.0014 s	0.0011 s
2000x2000	19,944	32,128,184	1,706,784	0.0024 s	0.0019 s

Table Q2-2: memory (sys.getsizeof, the dict figure includes its key tuples) and time for the updates.

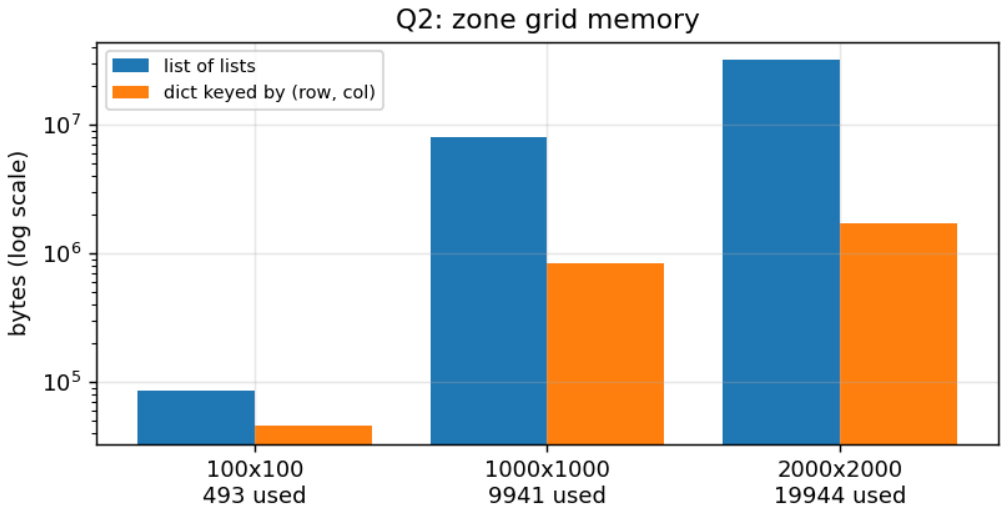


Figure Q2-2: memory of the two grids (log scale).

Time: reading or updating one zone is O(1) on average in both designs (an index twice for the list, one hash for the dictionary); the measured updates were about the same, the dictionary was a little faster because it does one lookup instead of two indexings. Space: the list of lists is O(rows x cols) whether or not anyone boards there (32,128,184 bytes for 2000 x 2000), the dictionary is O(k) for k non-empty zones (1,706,784

bytes for 19,944 zones, about 19 times less). The cost per used zone is about 86 bytes in the dictionary against 8 bytes per cell in the list, so from my numbers the dictionary only wins when fewer than about 1 in 10 zones is used; for a full grid the list of lists is better. Also, a dictionary cannot give "all zones of row 3" without searching all keys, while a list can give grid[3].

9. Conclusion

Use a tuple for a fixed record such as a (latitude, longitude) pair, a dictionary key, or a value returned from a function; use a list for a collection that grows or changes. Immutability protects the data from being changed by mistake by some other part of the program that holds the same object, and it makes a hash key safe. It does not protect an object that is inside the tuple (the list-in-tuple test), and it does not remove the aliasing bug for lists, which was the cause of the zone-grid problem: the developer saw every update in every row because all rows were one list.

Q3 - DICTIONARIES: FARMER HELPLINE COMPLAINT ANALYSIS (7 Marks)

1. Aim

To count the most frequent words in complaints (ignoring common words) with a dictionary, to group complaints by district and by crop, and to compare the dictionary loop with list.count() and collections.Counter for 1,000, 10,000 and 100,000 complaints.

2. Predictions made before running

- In the given example seeds = 3, price = 2, received = 2, and the stop words "not" and "high" are dropped, so the expected top 3 is [('seeds', 3), ('price', 2), ('received', 2)].
- The dictionary loop and Counter will take about the same time and grow in proportion to the number of words; list.count() for every distinct word will be slower, and the gap will grow with the number of distinct words d, not with the number of complaints alone.
- Counter and the dictionary loop will need memory only for the distinct words; list.count() needs the full list of words.

3. Program: Complete the Code (TODO 1 and TODO 2)

File: Q3/q3_complete_code.py. TODO 1 (frequencies) uses counts[word] = counts.get(word, 0) + 1 for every word not in the stop set. TODO 2 (top) sorts the items with key=lambda pair: (-pair[1], pair[0]) and takes the first k. The rest of the given program is unchanged.

```python
"""Q3 - Complete the Code (TODO 1 and TODO 2 completed, the rest of the given program unchanged)."""
calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}


def frequencies(texts, stop):
    # TODO 1: Return a dictionary word -> count for every word that is not in stop,
    # using dict.get()
    counts = {}
    for text in texts:
        for word in text.split():
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts


def top(counts, k):
    # TODO 2: Return the k most frequent (word, count) pairs, highest count first and
    # alphabetical for ties
    return sorted(counts.items(), key=lambda pair: (-pair[1], pair[0]))[:k]


print(top(frequencies(calls, STOP), 3))

# --- extra checks for the explanation question (not part of the given program) ---
counts = frequencies(calls, STOP)
print("all counts:", counts)
print("tied words in dictionary order :", [w for w, c in counts.items() if c == 2])
print("tied words after the sort key  :", [w for w, c in top(counts, 10) if c == 2])
```

4. Output of the Complete the Code program

Real run (command: python3 q3_complete_code.py), Python 3.10.12.



The first line is exactly the expected output [('seeds', 3), ('price', 2), ('received', 2)]. Why price comes before received although both occur twice: the sort key is (-count, word). The counts are equal (-2 and -2), so Python compares the second part, the words, and "price" is before "received" in alphabetical order. The third and fourth lines of my output show that this really matters: in the dictionary the word received was inserted first (it appears in the first call), so if I sorted by count only, the stable sort would keep that dictionary order and give received before price.

5. Program: the investigation

File: Q3/q3_complaints.py. It builds random complaints with a district, a crop and a text; counts with get(), with setdefault(), with list.count() and with Counter; groups by district and by crop with a dictionary of lists; tests empty text, upper/lower case and punctuation; and measures time and memory.

```python
"""Q3 - Kisan call centre: word counts, districts, crops, timings."""
import json
import random
import string
import sys
import time
from collections import Counter

STOP = {"not", "high", "the", "is", "a", "and", "to", "of", "in", "for", "my", "has", "have", "very"}
DISTRICTS = ["Thanjavur", "Madurai", "Salem", "Erode", "Vellore", "Coimbatore", "Tirunelveli", "Trichy"]
CROPS = ["paddy", "cotton", "sugarcane", "groundnut", "banana", "maize"]
VOCAB = ("seeds fertiliser price subsidy received delayed insurance pump electricity canal water loan "
         "pesticide market truck storage payment drought flood pest disease soil credit tractor wage "
         "repair bank form officer token weighing rate harvest sowing quality shortage").split()


def frequencies_get(texts, stop):
    counts = {}
    for text in texts:
        for word in clean(text):
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts


def frequencies_setdefault(texts, stop):
    counts = {}
    for text in texts:
        for word in clean(text):
            if word not in stop:
                counts.setdefault(word, 0)
                counts[word] += 1
    return counts


def clean(text):
    """lower-case, remove punctuation, split into words"""
    return text.lower().translate(str.maketrans("", "", string.punctuation)).split()


def frequencies_listcount(texts, stop):
    words = [w for t in texts for w in clean(t) if w not in stop]
    return {w: words.count(w) for w in set(words)}      # one full pass of the list per distinct word


def frequencies_counter(texts, stop):
```

```
    return Counter(w for t in texts for w in clean(t) if w not in stop)


def top(counts, k):
    return sorted(counts.items(), key=lambda p: (-p[1], p[0]))[:k]


def make_complaints(n, seed=11, vocab=VOCAB):
    rng = random.Random(seed)
    out = []
    for _ in range(n):
        words = [rng.choice(vocab) for _ in range(rng.randint(4, 8))] + ["not", "the"]
        rng.shuffle(words)
        out.append({"district": rng.choice(DISTRICTS), "crop": rng.choice(CROPS), "text": " ".join(words).capitalize() + "."})
    return out


def best(fn, repeat=3):
    b = 1e9
    for _ in range(repeat):
        t = time.perf_counter(); fn(); b = min(b, time.perf_counter() - t)
    return b


results = {}

# ---------- 1. small run: words, districts, crops ----------
small = make_complaints(12, seed=5)
print("A few complaints:")
for c in small[:4]:
    print("  ", c)
texts = [c["text"] for c in small]
print("\nTop 5 words (count desc, then A-Z):", top(frequencies_get(texts, STOP), 5))
print("get() and setdefault() give the same dictionary:", frequencies_get(texts, STOP) == frequencies_setdefault(texts, STOP))

per_district = {}
by_crop = {}
for c in small:
    per_district[c["district"]] = per_district.get(c["district"], 0) + 1
    by_crop.setdefault(c["crop"], []).append(c["text"])
print("\nComplaints per district:", dict(sorted(per_district.items())))
print("Complaints for crop 'paddy':", by_crop.get("paddy", []))
print("Crops and how many complaints each:", {k: len(v) for k, v in sorted(by_crop.items())})

# ---------- 2. views and order ----------
d = {"seeds": 3}
```

```
keys, values, items = d.keys(), d.values(), d.items()
d["price"] = 2
d["received"] = 2
print("\nViews are live. keys/values/items were taken BEFORE adding two words:")
print("  keys  :", list(keys))
print("  values:", list(values))
print("  items :", list(items))
print("Order = insertion order (Python 3.7+):", list(d))
del d["seeds"]
d["seeds"] = 9
print("after deleting and re-adding 'seeds' it moves to the end:", list(d))

# ---------- 3. edge cases ----------
print("\nEdge cases")
tests = {
    "empty text": [""],
    "no complaints": [],
    "upper/lower case": ["Seeds SEEDS seeds"],
    "punctuation": ["Seeds, seeds! (seeds) price."],
    "only stop words": ["not high the"],
}
for name, t in tests.items():
    r = [f(t, STOP) for f in (frequencies_get, frequencies_listcount, frequencies_counter)]
    print("  %-17s -> %s   all three methods agree: %s" % (name, dict(r[0]), r[0] == r[1] == dict(r[2])))
print("  without clean(), plain split() on 'Seeds, seeds!' gives:", "Seeds, seeds!".split())

# ---------- 4. timings ----------
print("\nTimings in seconds (best of 3). Vocabulary of %d words. Each complaint has 6-10 words." % len(VOCAB))
print("%9s %9s %11s %11s %11s %13s" % ("complaints", "words", "dict get()", "setdefault", "Counter", "list.count()"))
rows = []
for n in (1000, 10000, 100000):
    comp = make_complaints(n)
    texts = [c["text"] for c in comp]
    nw = sum(len(clean(t)) for t in texts)
    t_get = best(lambda: frequencies_get(texts, STOP))
    t_sd = best(lambda: frequencies_setdefault(texts, STOP))
    t_ct = best(lambda: frequencies_counter(texts, STOP))
    t_lc = best(lambda: frequencies_listcount(texts, STOP), 1 if n == 100000 else 3)
    same = frequencies_get(texts, STOP) == frequencies_listcount(texts, STOP) == dict(frequencies_counter(texts, STOP))
    rows.append([n, nw, t_get, t_sd, t_ct, t_lc])
    print("%9d %9d %11.5f %11.5f %11.5f %13.5f   same result: %s" % (n, nw, t_get, t_sd, t_ct, t_lc, same))
results["times"] = rows

# ---------- 5. effect of d (distinct words) ----------
print("\nEffect of the number of distinct words d (n = 20,000 complaints, about 125,000 words)")
print("%8s %12s %14s %14s" % ("d", "dict get()", "list.count()", "count/dict"))
```

```
drows = []
for dsize in (10, 100, 1000):
    vocab = ["w%d" % i for i in range(dsize)]
    texts = [c["text"] for c in make_complaints(20000, vocab=vocab)]
    t1 = best(lambda: frequencies_get(texts, STOP), 2)
    t2 = best(lambda: frequencies_listcount(texts, STOP), 1)
    drows.append([dsize, t1, t2])
    print("%8d %12.5f %14.5f %14.1f" % (dsize, t1, t2, t2 / t1))
results["dtest"] = drows

# ---------- 6. memory ----------
comp = make_complaints(100000)
texts = [c["text"] for c in comp]
cnt = frequencies_get(texts, STOP)
cc = frequencies_counter(texts, STOP)
words_list = [w for t in texts for w in clean(t) if w not in STOP]
print("\nMemory (sys.getsizeof)")
print("  dict of %d distinct words: %d B | Counter: %d B | list of all %d words (list.count needs it): %d B"
      % (len(cnt), sys.getsizeof(cnt), sys.getsizeof(cc), len(words_list), sys.getsizeof(words_list)))
results["mem"] = [len(cnt), sys.getsizeof(cnt), sys.getsizeof(cc), len(words_list), sys.getsizeof(words_list)]
print("  Top 5 of 100,000 complaints:", top(cnt, 5))
print("  Counter.most_common(5)     :", cc.most_common(5), "(ties are NOT sorted A-Z, so I still sort myself)")

with open("results_q3.json", "w") as f:
    json.dump(results, f)
```

6. Output of the investigation

Real run (command: python3 q3_complaints.py), Python 3.10.12.

```
Terminal - python3
$ python3 q3_complaints.py
A few complaints:
   {'district': 'Thanjavur', 'crop': 'maize', 'text': 'Not token sowing payment fertiliser the subsidy water
    storage credit.'}
   {'district': 'Thanjavur', 'crop': 'paddy', 'text': 'Water loan not wage drought bank the.'}
   {'district': 'Tirunelveli', 'crop': 'sugarcane', 'text': 'Flood market water not the disease water.'}
   {'district': 'Coimbatore', 'crop': 'paddy', 'text': 'Canal not bank the tractor water.'}

Top 5 words (count desc, then A-Z): [('water', 6), ('loan', 5), ('rate', 5), ('bank', 4), ('canal', 4)]
get() and setdefault() give the same dictionary: True

Complaints per district: {'Coimbatore': 3, 'Salem': 1, 'Thanjavur': 3, 'Tirunelveli': 3, 'Trichy': 1, 'Vellore':
     1}
Complaints for crop 'paddy': ['Water loan not wage drought bank the.', 'Canal not bank the tractor water.',
    'Disease not loan weighing credit pest weighing the.', 'Rate not market canal bank price soil the subsidy.']
Crops and how many complaints each: {'banana': 1, 'maize': 2, 'paddy': 4, 'sugarcane': 5}

Views are live. keys/values/items were taken BEFORE adding two words:
  keys  : ['seeds', 'price', 'received']
  values: [3, 2, 2]
  items : [('seeds', 3), ('price', 2), ('received', 2)]
Order = insertion order (Python 3.7+): ['seeds', 'price', 'received']
after deleting and re-adding 'seeds' it moves to the end: ['price', 'received', 'seeds']

Edge cases
  empty text        -> {}   all three methods agree: True
  no complaints     -> {}   all three methods agree: True
  upper/lower case  -> {'seeds': 3}   all three methods agree: True
  punctuation       -> {'seeds': 3, 'price': 1}   all three methods agree: True
  only stop words   -> {}   all three methods agree: True
  without clean(), plain split() on 'Seeds, seeds!' gives: ['Seeds,', 'seeds!']

Timings in seconds (best of 3). Vocabulary of 36 words. Each complaint has 6-10 words.
complaints     words  dict get()  setdefault     Counter  list.count()
     1000      8001     0.00235     0.00249     0.00232       0.00387   same result: True
```

```
Terminal - python3
  ... (output continues)
    10000     80286     0.02437     0.02533     0.02358       0.04061   same result: True
   100000    800570     0.24444     0.25581     0.23808       0.42146   same result: True

Effect of the number of distinct words d (n = 20,000 complaints, about 125,000 words)
       d   dict get()   list.count()     count/dict
      10      0.03925        0.04622            1.2
     100      0.04187        0.13145            3.1
    1000      0.04367        0.96040           22.0

Memory (sys.getsizeof)
  dict of 36 distinct words: 1176 B | Counter: 1192 B | list of all 600570 words (list.count needs it): 5274392
    B
  Top 5 of 100,000 complaints: [('electricity', 16946), ('storage', 16936), ('shortage', 16876), ('credit',
    16841), ('pesticide', 16809)]
  Counter.most_common(5)     : [('electricity', 16946), ('storage', 16936), ('shortage', 16876), ('credit',
    16841), ('pesticide', 16809)] (ties are NOT sorted A-Z, so I still sort myself)
```

7. Analysis

- **How a dictionary finds a key.** A dictionary is a hash table. Python computes hash(key), uses it to pick a slot in an internal array and compares the key stored there; if it is the same key the value is returned, otherwise it probes the next slot. The work is one hash plus a few comparisons, and the table is resized to stay mostly empty, so the average cost does not depend on how many keys are stored. That is why a

dictionary with 36 words and one with a million words answer in about the same time (the worst case O(n) only happens when many keys collide, which is rare).

- **Dictionary against list.count().** Counting with a dictionary does one lookup and one store per word, so n words cost O(n). Counting with list.count() for every distinct word scans the whole list of n words once for each of the d distinct words, so O(n x d). My d test (Table Q3-2) shows this clearly: with n fixed, going from 10 to 1,000 distinct words made the dictionary only 1.1 times slower but list.count() 21 times slower (0.046 s to 0.960 s).
- **Order of keys.** Since Python 3.7 a dictionary keeps the order in which keys were first inserted (it is part of the language, not an accident of one implementation). Changing a value does not change the order; deleting a key and adding it again puts it at the end (my test: seeds moved to the end). The order is insertion order, not sorted order, so for a ranking I still need sorted().
- **Key, value and item views.** keys(), values() and items() return views, not copies. A view shows the dictionary as it is now: in my test I took the three views and then added two words, and the views showed the new words (the printed lists had all three). items() gives (key, value) pairs, which is what top() sorts.
- **Tests.** Edge cases: an empty text and an empty list of complaints give {}; "Seeds SEEDS seeds" gives {'seeds': 3} because of lower(); "Seeds, seeds! (seeds) price." gives seeds 3 and price 1 only because I removed the punctuation first (plain split() would have given "Seeds," and "seeds!" as different words). A text with only stop words gives {}. The dictionary, list.count() and Counter versions agreed in every case and on all three big inputs.

complaints	words	dict + get()	dict + setdefault()	Counter	list.count() per word
1,000	8,001	0.00235 s	0.00249 s	0.00232 s	0.00387 s
10,000	80,286	0.02437 s	0.02533 s	0.02358 s	0.04061 s
100,000	800,570	0.24444 s	0.25581 s	0.23808 s	0.42146 s

Table Q3-1: time to count the words, best of 3 (list.count() best of 1 at 100,000). 36 distinct words in the vocabulary.

distinct words d	dict + get()	list.count() per word	list.count() / dict
10	0.03925 s	0.04622 s	1.2
100	0.04187 s	0.13145 s	3.1
1,000	0.04367 s	0.96040 s	22.0

Table Q3-2: n fixed (20,000 complaints, about 125,000 words), d changes.

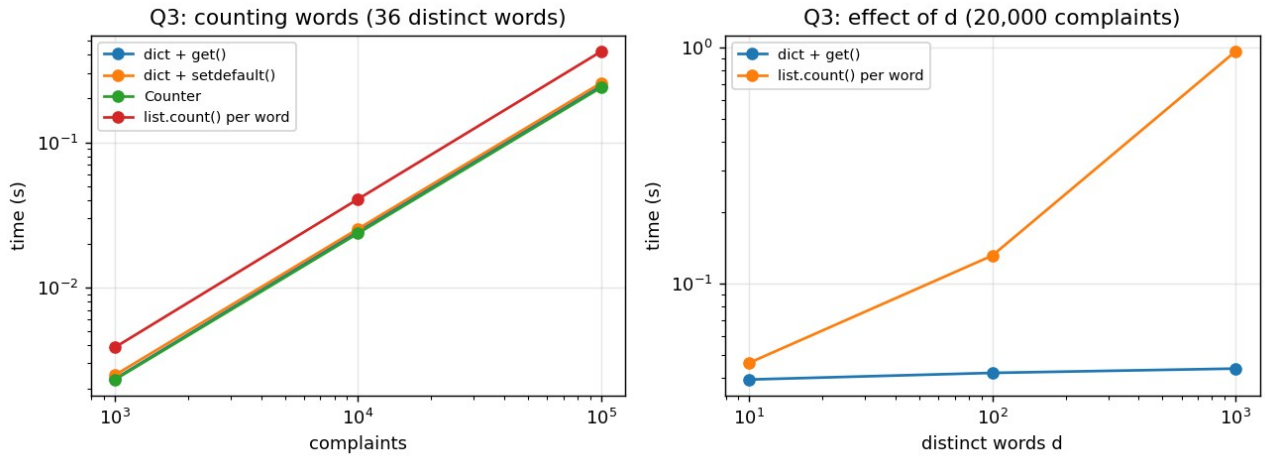


Figure Q3-1: time against the number of complaints (left) and against the number of distinct words (right), log scales.

The first prediction held for the dictionary and Counter: they differ by less than 5% (0.244 s and 0.238 s for 100,000 complaints) and both grow 10 times for 10 times more complaints. list.count() was 1.7 times slower than the dictionary at d = 36 but will be far worse for a real vocabulary of thousands of words. Memory at 100,000 complaints: the dictionary of 36 distinct words is 1,176 bytes, the Counter 1,192 bytes, but the list of all 600,570 words that list.count() needs is 5,274,392 bytes (about 4,500 times more). Counter.most_common(5) gave the same top 5 as my sorted() here, but Counter does not sort ties alphabetically (the question wants that), so I still need my own key for the ties.

8. Engineering Decision

For 100,000 complaints a day I would deploy the dictionary-based count, written as collections.Counter over the cleaned words, followed by my sorted(..., key=lambda p: (-p[1], p[0])) for the top k. It is O(n), took 0.24 s for 100,000 complaints on my computer, uses memory only for the distinct words, and is the shortest code to maintain. list.count() should not be used: its cost is n x d, and a real helpline vocabulary has thousands of distinct words, so the day batch would take minutes instead of a fraction of a second.

9. Alternative Concept and Complexity Analysis

list.count() version: first put all words in one list, then for each distinct word w call words.count(w) (frequencies_listcount in my program). Counter version: Counter(words) does the same one-pass counting as my loop but inside C code (collections._count_elements).

Method	Time for n words, d distinct	Extra space	Measured at 100,000 complaints
dictionary loop (get)	O(n)	O(d)	0.2444 s
dictionary loop (setdefault)	O(n)	O(d)	0.2558 s
Counter	O(n)	O(d)	0.2381 s
list.count() for every distinct word	O(n x d)	O(n) for the word list + O(d)	0.4215 s

Table Q3-3: complexity and measurement.

The measurements agree with the analysis: the dictionary and Counter are O(n) and the list.count() method is O(n x d), which I confirmed by changing d (Table Q3-2: 1.2, 3.1 and 22.0 times slower for d = 10, 100,

1000). Space is O(d) for the dictionary and Counter (only the distinct words) but the list.count() method also needs the whole word list, O(n).

10. Conclusion

A dictionary is the natural tool for counting because "have I seen this word, and how many times?" is exactly what a hash lookup answers in constant time. Grouping by district and by crop used the same idea with setdefault(key, []).append(...), a dictionary of lists. The part I would have got wrong without testing was the tie order in the top-k and the punctuation: both needed explicit code.

Q4 - SETS: ELECTIVE REGISTRATION CLASH DETECTION (7 Marks)

1. Aim

To keep each elective's register numbers in a set and use union, intersection, difference and symmetric difference to find the clash students (in both electives), the students in exactly one and those in none, to draw a Venn diagram for each result, and to measure sets against lists for 10,000, 100,000 and 1,000,000 register numbers.

2. Predictions made before running

- For the given data, both = {192311004, 192311009}, exactly one = {001, 002, 007, 010} and neither = {003, 005, 006, 008}, so 2 + 4 + 4 = 10 students.
- x in set will take the same time for any size (about a fraction of a microsecond), and x in list will grow in proportion to the length of the list.
- The nested loops will take about 4 times longer each time n doubles (O(n x m)), while the set & grows about 2 times.
- The set uses more memory than a list of the same strings, because it has a hash table.

3. Program: Complete the Code (TODO 1)

File: Q4/q4_complete_code.py. TODO 1 uses set operators only: both = ai_elective & iot_elective, exactly_one = ai_elective ^ iot_elective, neither = all_students - (ai_elective | iot_elective). Below it I rewrote the same with the methods intersection, symmetric_difference, difference and union, and compared the results.

```
"""Q4 - Complete the Code (TODO 1 completed using set operators only, rest unchanged)."""
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}

# TODO 1: Use set operators to find: students in both electives, students in exactly
# one, and students in neither
both = ai_elective & iot_elective                              # intersection
exactly_one = ai_elective ^ iot_elective                       # symmetric difference
neither = all_students - (ai_elective | iot_elective)          # difference from the union

print("clash :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither :", sorted(neither))

# --- Rewrite TODO 1 using set methods and check that the results are the same ---
both_m = ai_elective.intersection(iot_elective)
exactly_one_m = ai_elective.symmetric_difference(iot_elective)
neither_m = all_students.difference(ai_elective.union(iot_elective))
print("\nSame with methods:")
print("clash :", sorted(both_m))
print("exactly one:", sorted(exactly_one_m))
print("neither :", sorted(neither_m))
print("operators and methods give the same results:",
      both == both_m and exactly_one == exactly_one_m and neither == neither_m)
```

4. Output of the Complete the Code program

Real run (command: python3 q4_complete_code.py), Python 3.10.12.

```
Terminal - python3

$ python3 q4_complete_code.py
clash : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither : ['192311003', '192311005', '192311006', '192311008']

Same with methods:
clash : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither : ['192311003', '192311005', '192311006', '192311008']
operators and methods give the same results: True
```

The first three lines match the expected output of the question exactly (clash 004 and 009; exactly one 001, 002, 007, 010; neither 003, 005, 006, 008), and the last line says True: the operator version and the method version give identical results.

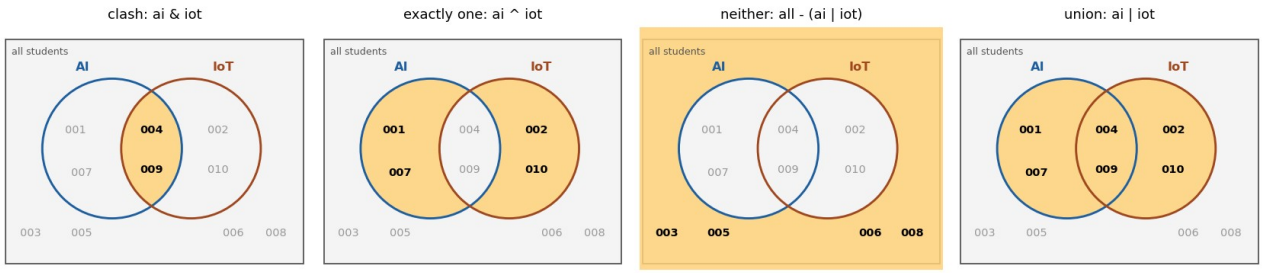


Figure Q4-1: Venn diagrams of the four results (shaded = the result; bold numbers are the last three digits of the register numbers in the result).

5. Program: the investigation

File: Q4/q4_electives.py. It removes duplicate registrations, verifies a small case by hand, tests empty and identical electives, and measures x in list against x in set and the common-student search with nested loops, a sorted walk and the set intersection.

```python
"""Q4 - Elective registration clash detection: sets vs lists, measurements."""
import json
import random
import sys
import time


def best(fn, repeat=3):
    b = 1e9
    for _ in range(repeat):
        t = time.perf_counter(); fn(); b = min(b, time.perf_counter() - t)
    return b


def regs(n, seed=1):
    """n distinct register numbers as strings (9 digits)"""
    rng = random.Random(seed)
    return [str(x) for x in rng.sample(range(192300000, 192300000 + 3 * n), n)]


def common_nested(a, b):
    out = []
    for x in a:
        for y in b:
            if x == y:
                out.append(x)
                break
    return out


def common_sorted(a, b):
    """sort both lists, then walk through them together"""
    a, b = sorted(a), sorted(b)
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            out.append(a[i]); i += 1; j += 1
        elif a[i] < b[j]:
            i += 1
        else:
            j += 1
    return out


results = {}
```

```
# ---------- 1. duplicates and hand-checked small case ----------
raw = ["192311001", "192311004", "192311001", "192311007", "192311004", "192311001"]
print("Registrations typed in with repeats:", raw)
print("set(raw) removes duplicates      :", sorted(set(raw)), "-> %d people, %d registrations" % (len(set(raw)), len(raw)))
print("list.count shows how many times each: ", {x: raw.count(x) for x in sorted(set(raw))})

ai = {"A", "B", "C", "D"}
iot = {"C", "D", "E"}
everyone = {"A", "B", "C", "D", "E", "F"}
print("\nSmall case by hand: AI = {A,B,C,D}, IoT = {C,D,E}, all = {A..F}")
print("  by hand -> both {C, D}; exactly one {A, B, E}; neither {F}; union {A,B,C,D,E}")
print("  program -> both", sorted(ai & iot), "| exactly one", sorted(ai ^ iot), "| neither", sorted(everyone - (ai | iot)), "| union", sorted(ai | iot))
print("  only in AI (difference):", sorted(ai - iot), "| only in IoT:", sorted(iot - ai))
print("  counts: |AI|=%d |IoT|=%d |both|=%d |union|=%d  and  |AI|+|IoT|-|both| = %d" % (len(ai), len(iot), len(ai & iot), len(ai | iot), len(ai) + len(iot) - len(ai & iot)))

# ---------- 2. edge cases ----------
print("\nEdge cases")
e = set()
x = {"1", "2", "3"}
print("  empty elective & full      :", e & x, "| union:", sorted(e | x), "| exactly one:", sorted(e ^ x))
print("  both electives empty       :", e & e, e | e, e ^ e)
print("  identical electives        : both =", sorted(x & x), "| exactly one =", sorted(x ^ x), "| neither =", sorted({"1", "2", "3", "4"} - (x | x)))
try:
    {[1, 2], [3]}
except TypeError as err:
    print("  set of lists               : TypeError:", err)
print("  set of tuples              :", {(13.04, 80.23), (13.08, 80.27), (13.04, 80.23)})
print("  set of strings with repeats:", {"a", "a", "b"})

# ---------- 3. x in list vs x in set ----------
print("\nMembership test, 200 lookups of a register number that is NOT there (worst case for a list)")
print("%10s %14s %14s %12s %14s %14s" % ("n", "x in list", "x in set", "ratio", "list bytes", "set bytes"))
mem_rows = []
for n in (10_000, 100_000, 1_000_000):
    L = regs(n)
    S = set(L)
    missing = "999999999"
    reps = 200 if n < 1_000_000 else 100
    t0 = time.perf_counter()
    for _ in range(reps):
        missing in L
    tl = (time.perf_counter() - t0) / reps
    t0 = time.perf_counter()
    for _ in range(reps * 100):
        missing in S
```

```
    ts = (time.perf_counter() - t0) / (reps * 100)
    tbuild = best(lambda: set(L), 2)
    mem_rows.append([n, tl, ts, sys.getsizeof(L), sys.getsizeof(S), tbuild])
    print("%10d %12.2f us %12.3f us %12.0f %14d %14d" % (n, tl * 1e6, ts * 1e6, tl / ts, sys.getsizeof(L), sys.getsizeof(S)))
results["member"] = mem_rows
print("  (set(L) itself takes: %s)" % ", ".join("%.4f s for n=%d" % (r[5], r[0]) for r in mem_rows))

# ---------- 4. common students: nested loops vs & vs sorted walk ----------
print("\nFinding the common students (about half of the two electives overlap)")
print("%10s %14s %14s %14s %10s" % ("n each", "nested loops", "sorted walk", "set  &", "common"))
com_rows = []
for n in (1000, 2000, 4000, 8000):
    pool = regs(int(n * 1.5), seed=n)
    a, b = pool[:n], pool[n // 2: n // 2 + n]
    t_n = best(lambda: common_nested(a, b), 1)
    t_s = best(lambda: common_sorted(a, b))
    t_i = best(lambda: set(a) & set(b))
    ok = sorted(common_nested(a, b)) == common_sorted(a, b) == sorted(set(a) & set(b))
    com_rows.append([n, t_n, t_s, t_i])
    print("%10d %14.5f %14.5f %14.6f %10d   same: %s" % (n, t_n, t_s, t_i, len(set(a) & set(b)), ok))
print("  and with the set only (nested loops would take hours here):")
big_rows = []
for n in (10_000, 100_000, 1_000_000):
    pool = regs(int(n * 1.5), seed=n)
    a, b = pool[:n], pool[n // 2: n // 2 + n]
    t_s = best(lambda: common_sorted(a, b), 1)
    t_i = best(lambda: set(a) & set(b), 2)
    ok = common_sorted(a, b) == sorted(set(a) & set(b))
    big_rows.append([n, t_s, t_i])
    print("%10d   sorted walk %.4f s | set & %.4f s | same: %s" % (n, t_s, t_i, ok))
results["common"] = com_rows
results["common_big"] = big_rows

# ---------- 5. memory of the two ways ----------
a = regs(100_000)
print("\nMemory for 100,000 register numbers: list %d B, set %d B (strings themselves are the same objects in both)" % (sys.getsizeof(a), sys.getsizeof(set(a))))

with open("results_q4.json", "w") as f:
    json.dump(results, f)
```

6. Output of the investigation

Real run (command: python3 q4_electives.py), Python 3.10.12.

```
Terminal - python3
$ python3 q4_electives.py
Registrations typed in with repeats: ['192311001', '192311004', '192311001', '192311007', '192311004',
    '192311001']
set(raw) removes duplicates      : ['192311001', '192311004', '192311007'] -> 3 people, 6 registrations
list.count shows how many times each:  {'192311001': 3, '192311004': 2, '192311007': 1}

Small case by hand: AI = {A,B,C,D}, IoT = {C,D,E}, all = {A..F}
  by hand -> both {C, D}; exactly one {A, B, E}; neither {F}; union {A,B,C,D,E}
  program -> both ['C', 'D'] | exactly one ['A', 'B', 'E'] | neither ['F'] | union ['A', 'B', 'C', 'D', 'E']
  only in AI (difference): ['A', 'B'] | only in IoT: ['E']
  counts: |AI|=4 |IoT|=3 |both|=2 |union|=5  and  |AI|+|IoT|-|both| = 5

Edge cases
  empty elective & full      : set() | union: ['1', '2', '3'] | exactly one: ['1', '2', '3']
  both electives empty       : set() set() set()
  identical electives        : both = ['1', '2', '3'] | exactly one = [] | neither = ['4']
  set of lists               : TypeError: unhashable type: 'list'
  set of tuples              : {(13.04, 80.23), (13.08, 80.27)}
  set of strings with repeats: {'a', 'b'}

Membership test, 200 lookups of a register number that is NOT there (worst case for a list)
         n      x in list       x in set        ratio     list bytes      set bytes
     10000        56.13 us        0.040 us         1409          85176         524504
    100000       567.48 us        0.033 us        17150         800984        4194520
   1000000      5693.49 us        0.032 us       180483        8448728       33554648
  (set(L) itself takes: 0.0005 s for n=10000, 0.0047 s for n=100000, 0.0547 s for n=1000000)

Finding the common students (about half of the two electives overlap)
    n each   nested loops    sorted walk         set  &     common
      1000        0.01013        0.00037       0.000063        500   same: True
      2000        0.04090        0.00077       0.000164       1000   same: True
      4000        0.16274        0.00163       0.000284       2000   same: True
      8000        0.65266        0.00348       0.000684       4000   same: True
  and with the set only (nested loops would take hours here):
     10000   sorted walk 0.0045 s | set & 0.0010 s | same: True
```

```
Terminal - python3
  ... (output continues)
    100000   sorted walk 0.0594 s | set & 0.0105 s | same: True
   1000000   sorted walk 0.8079 s | set & 0.1636 s | same: True

Memory for 100,000 register numbers: list 800984 B, set 4194520 B (strings themselves are the same objects in
    both)
```

7. Analysis

- **Why duplicates disappear.** A set stores each element once. When I add something that is equal to an element already inside, the set finds the existing one through its hash and equality test and does not add another. So set(raw) turned 6 registrations with repeats into 3 students. The set also stores no counts, so if I need to know how many times, I use list.count() or a dictionary before converting.
- **Why membership is faster for a set.** x in list compares x with the items one after another until it finds it, which is O(n) (and the whole list when the number is absent). x in set computes the hash of x and goes to one place in the table, O(1) on average. For a number that is NOT there, the list took 56.1 us for 10,000 numbers, 567.5 us for 100,000 and 5693.5 us for 1,000,000, growing in proportion to n, while the set took about 0.03 us at every size. At 1,000,000 numbers the set was about 180,483 times faster.

- **Lists and tuples in sets.** A set must be able to hash each element, and the hash must not change. A list can change, so it is unhashable (the program printed TypeError: unhashable type: 'list'). A tuple of numbers cannot change, so it is hashable and (13.04, 80.23) can be stored, and a repeated tuple was stored once.
- **Verification by hand.** Hand check: AI = {A,B,C,D}, IoT = {C,D,E}, all = {A..F}. Both = {C, D}, exactly one = {A, B, E}, neither = {F}, union = {A, B, C, D, E}. The program printed the same. As a second check, |AI| + |IoT| - |both| = 4 + 3 - 2 = 5 = |union|.
- **Empty and identical electives.** If an elective is empty, the clash set is empty and "exactly one" is the other elective. If both electives are the same, the clash is the whole set, "exactly one" is empty and "neither" is the students outside it. Both worked.

n	x in list (absent)	x in set (absent)	list / set	list object (B)	set object (B)	set(list) build time
10,000	56.1 us	0.040 us	1,409	85,176	524,504	0.0005 s
100,000	567.5 us	0.033 us	17,150	800,984	4,194,520	0.0047 s
1,000,000	5693.5 us	0.032 us	180,483	8,448,728	33,554,648	0.0547 s

Table Q4-1: membership test (average of 200 or 100 lookups of a number that is not present). Memory is sys.getsizeof of the container only.

n in each elective	nested loops	sort both and walk	set &	common students
1,000	0.01013 s	0.00037 s	0.00006 s	500
2,000	0.04090 s	0.00077 s	0.00016 s	1,000
4,000	0.16274 s	0.00163 s	0.00028 s	2,000
8,000	0.65266 s	0.00348 s	0.00068 s	4,000
10,000	not run	0.00445 s	0.00096 s	5,000
100,000	not run	0.05941 s	0.01055 s	50,000
1,000,000	not run	0.80786 s	0.16355 s	500,000

Table Q4-2: finding the common students (half of each elective is in both). "set &" includes building both sets.

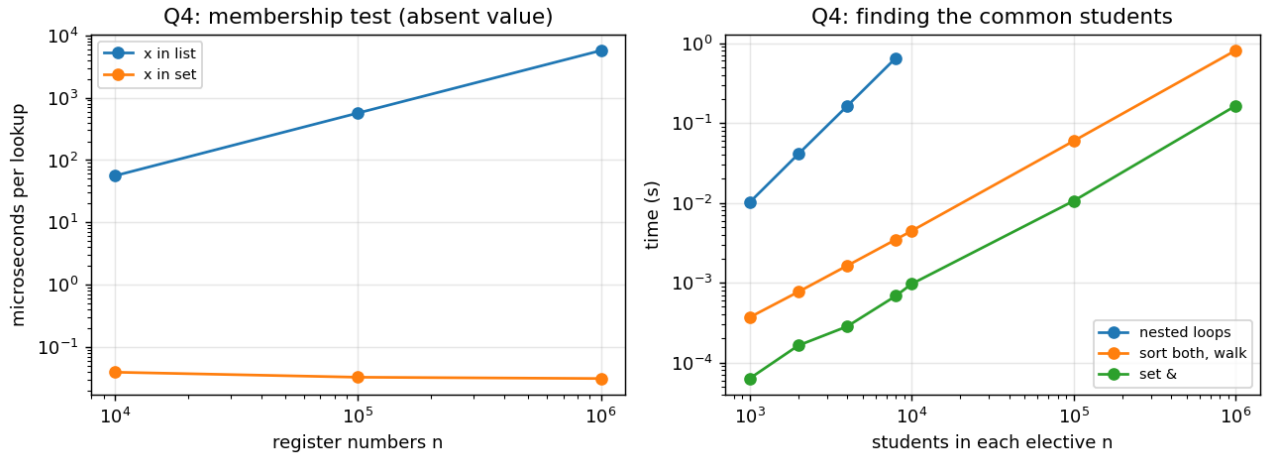


Figure Q4-2: membership test and common-student search (log scales).

The predictions held. Each time n doubled, the nested loops took about 4 times longer (0.01013 s, 0.04090 s, 0.16274 s, 0.65266 s), while the set & took less than 3 times longer each time. At n = 8,000 the set was about 954 times faster than the nested loops. I did not run nested loops at 10,000 and above for the big

sizes of the question. From the n-squared growth, 1,000,000 students would need about 2.8 hours (an estimate, not measured), but the set & did it in 0.1636 s. Memory: for 100,000 register numbers the list object was 800,984 bytes and the set 4,194,520 bytes (5.2 times more; the strings themselves are the same objects), so I pay memory to get speed.

8. Conclusion

I would convert a list into a set before processing when I need many membership tests, when I need to remove duplicates, or when I need union, intersection or difference. For example, checking each of 50,000 students against a set of clashes is O(50,000) but against a list it is O(50,000 x length). What is lost: the order of the items (a set has no position, no index or slicing), the duplicates and therefore the counts, and the ability to hold unhashable items such as lists. If I need the order I keep the list and make the set only for the lookups.

9. Alternative Concept and Complexity Analysis

Lists with nested loops: for each x in list A, scan list B (common_nested). Sorted lists: sort both lists, then walk through them together with two indexes, moving the smaller one forward (common_sorted). Sets: set(a) & set(b). All three gave the same common students in every test.

Method	Time	Extra space	Measured n = 8,000 / 1,000,000
nested loops	O(n x m)	O(1)	0.6527 s / not run
sort both, walk	O(n log n + m log m)	O(n + m) for the sorted copies	0.0035 s / 0.8079 s
set &	O(n + m) on average	O(n + m), bigger constant	0.0007 s / 0.1636 s

Table Q4-3: complexity and measurement.

The sorted walk is a good middle way when the data is already sorted or when memory is tight (a set uses about 5 times the memory of the list), because after sorting it only needs O(n + m) steps. The set is the fastest and the shortest to write, but its time is "on average" O(n + m): many hash collisions could make it slower.

Q5 - LIST COMPREHENSION: CLEANING SENSOR READINGS (6 Marks)

1. Aim

To clean cold-storage temperature readings (a reading above 50 or below -50 degrees C is a sensor error), convert the valid ones to Fahrenheit and list the minutes when the temperature was outside 2-8 degrees C, in three ways: a for loop with append, list comprehensions, and map() with filter() and lambda; alarm minutes with enumerate(). Then to compare time and memory for 10,000, 100,000 and 1,000,000 readings.

2. Predictions made before running

- Valid means -50 <= t <= 50, so 50.0 and -50.0 are valid but 50.1 is an error; an alarm means t < 2 or t > 8, so 2.0 and 8.0 are fine and 8.1 is an alarm. A reading such as 50.0 is valid AND an alarm.
- All three versions must give identical lists. The comprehension should be the fastest, map/filter with lambda the slowest (a Python function call for every item).
- A generator expression will use a tiny fixed memory, while a list comprehension of 1,000,000 numbers will need about 8 MB for the references.

3. Program

File: Q5/q5_sensors.py. The three versions are v_loop, v_comp and v_map. The alarm minutes are the positions in the original list, found with enumerate, so the minute number stays correct even when sensor errors are removed before it.

```python
"""Q5 - Cold-storage temperature monitor: loop vs comprehension vs map/filter."""
import json
import random
import sys
import time
import tracemalloc


def make_raw(n, seed=3, error_rate=0.03):
    """minute-by-minute readings in Celsius: mostly 2-8 C, some drift, some sensor errors"""
    rng = random.Random(seed)
    out = []
    for _ in range(n):
        x = rng.random()
        if x < error_rate:
            out.append(rng.choice([-1, 1]) * round(rng.uniform(50.5, 99.9), 1))   # sensor error
        elif x < 0.12:
            out.append(round(rng.choice([rng.uniform(-4, 1.9), rng.uniform(8.1, 14)]), 1))  # real alarm
        else:
            out.append(round(rng.uniform(2, 8), 1))
    return out


def to_f(c):
    return c * 9 / 5 + 32


# ---------- version 1: for loop with append ----------
def v_loop(raw):
    celsius, fahrenheit, alarms = [], [], []
    for minute, t in enumerate(raw):
        if -50 <= t <= 50:
            celsius.append(t)
            fahrenheit.append(to_f(t))
            if t < 2 or t > 8:
                alarms.append(minute)
    return celsius, fahrenheit, alarms


# ---------- version 2: list comprehensions with conditions ----------
def v_comp(raw):
    celsius = [t for t in raw if -50 <= t <= 50]
    fahrenheit = [c * 9 / 5 + 32 for c in celsius]
    alarms = [minute for minute, t in enumerate(raw) if -50 <= t <= 50 and not 2 <= t <= 8]
    return celsius, fahrenheit, alarms
```

```python
# ---------- version 3: map() and filter() with lambda ----------
def v_map(raw):
    celsius = list(filter(lambda t: -50 <= t <= 50, raw))
    fahrenheit = list(map(lambda c: c * 9 / 5 + 32, celsius))
    alarms = [m for m, t in filter(lambda p: -50 <= p[1] <= 50 and not 2 <= p[1] <= 8, enumerate(raw))]
    return celsius, fahrenheit, alarms


def best(fn, *a, repeat=3):
    b = 1e9
    for _ in range(repeat):
        t = time.perf_counter(); fn(*a); b = min(b, time.perf_counter() - t)
    return b


results = {}

# ---------- 1. small run ----------
raw = [4.5, 3.1, 120.0, 9.2, 5.0, -63.5, 1.5, 7.9, 2.0, 8.0, 8.1, 50.0, -50.0, 50.1]
print("raw readings :", raw)
for name, fn in (("for loop", v_loop), ("comprehension", v_comp), ("map/filter", v_map)):
    c, f, a = fn(raw)
    print("%-14s valid %d  alarm minutes %s" % (name, len(c), a))
c, f, a = v_comp(raw)
print("valid Celsius   :", c)
print("valid Fahrenheit:", [round(x, 1) for x in f])
print("alarm minutes (index in the original list, enumerate): ", a)
print("all three versions identical:", v_loop(raw) == v_comp(raw) == v_map(raw))
print("boundary check: 2.0 and 8.0 are NOT alarms, 8.1 is; 50.0 and -50.0 are valid, 50.1 is an error")

# ---------- 2. dict / set comprehension, generator ----------
print("\nDictionary, set comprehension and generator expression")
by_minute = {m: t for m, t in enumerate(raw) if -50 <= t <= 50 and not 2 <= t <= 8}
print("  dict comprehension {minute: temperature} of alarms:", by_minute)
print("  set comprehension of distinct valid whole degrees  :", sorted({int(t) for t in raw if -50 <= t <= 50}))
g = (t for t in raw if -50 <= t <= 50)
print("  generator expression:", g.__class__.__name__, "| first value:", next(g), "| second:", next(g))
print("  list(g) now gives only what is left:", list(g)[:3], "... (a generator can be used once)")

# ---------- 3. all valid / all invalid / empty ----------
print("\nEdge cases")
cases = {
    "all valid": [3.0, 5.5, 7.9, 2.0, 8.0],
    "all invalid": [51.0, -50.1, 99.0, -99.0],
    "empty": [],
```

```
}
for name, data in cases.items():
    r = v_loop(data), v_comp(data), v_map(data)
    print("  %-12s -> valid %d, alarms %s, three versions same: %s" % (name, len(r[1][0]), r[1][2], r[0] == r[1] == r[2]))
big_valid = [5.0] * 1_000_000
big_bad = [99.0] * 1_000_000
print("  1,000,000 all valid  : kept", len(v_comp(big_valid)[0]), "| alarms", len(v_comp(big_valid)[2]))
print("  1,000,000 all invalid: kept", len(v_comp(big_bad)[0]), "| alarms", len(v_comp(big_bad)[2]))

# ---------- 4. timings ----------
print("\nTimings in seconds (best of 3): clean + convert + alarm minutes")
print("%10s %12s %14s %12s" % ("readings", "for loop", "comprehension", "map/filter"))
rows = []
for n in (10_000, 100_000, 1_000_000):
    raw = make_raw(n)
    t1, t2, t3 = best(v_loop, raw), best(v_comp, raw), best(v_map, raw)
    same = v_loop(raw) == v_comp(raw) == v_map(raw)
    rows.append([n, t1, t2, t3])
    print("%10d %12.5f %14.5f %12.5f   identical results: %s" % (n, t1, t2, t3, same))
results["times"] = rows

# ---------- 5. memory ----------
print("\nMemory of the valid Celsius readings (sys.getsizeof and tracemalloc peak)")
print("%10s %16s %18s %20s %20s" % ("readings", "list comp (B)", "generator (B)", "peak list sum (B)", "peak generator sum (B)"))
mrows = []
for n in (10_000, 100_000, 1_000_000):
    raw = make_raw(n)
    lc = [t for t in raw if -50 <= t <= 50]
    ge = (t for t in raw if -50 <= t <= 50)
    tracemalloc.start()
    s1 = sum([t for t in raw if -50 <= t <= 50])
    p1 = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()
    tracemalloc.start()
    s2 = sum(t for t in raw if -50 <= t <= 50)
    p2 = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()
    mrows.append([n, sys.getsizeof(lc), sys.getsizeof(ge), p1, p2])
    print("%10d %16d %18d %20d %20d   sums equal: %s" % (n, sys.getsizeof(lc), sys.getsizeof(ge), p1, p2, s1 == s2))
results["mem"] = mrows

print("\nTime of sum(list comprehension) vs sum(generator) for 1,000,000 readings")
raw = make_raw(1_000_000)
tl = best(lambda: sum([t for t in raw if -50 <= t <= 50]))
tg = best(lambda: sum(t for t in raw if -50 <= t <= 50))
print("  sum([...]) %.5f s | sum(...) generator %.5f s" % (tl, tg))
```

```
results["sum"] = [tl, tg]

with open("results_q5.json", "w") as f:
    json.dump(results, f)
```

4. Output

Real run (command: python3 q5_sensors.py), Python 3.10.12.

```
Terminal - python3
$ python3 q5_sensors.py
raw readings : [4.5, 3.1, 120.0, 9.2, 5.0, -63.5, 1.5, 7.9, 2.0, 8.0, 8.1, 50.0, -50.0, 50.1]
for loop       valid 11  alarm minutes [3, 6, 10, 11, 12]
comprehension  valid 11  alarm minutes [3, 6, 10, 11, 12]
map/filter     valid 11  alarm minutes [3, 6, 10, 11, 12]
valid Celsius   : [4.5, 3.1, 9.2, 5.0, 1.5, 7.9, 2.0, 8.0, 8.1, 50.0, -50.0]
valid Fahrenheit: [40.1, 37.6, 48.6, 41.0, 34.7, 46.2, 35.6, 46.4, 46.6, 122.0, -58.0]
alarm minutes (index in the original list, enumerate):  [3, 6, 10, 11, 12]
all three versions identical: True
boundary check: 2.0 and 8.0 are NOT alarms, 8.1 is; 50.0 and -50.0 are valid, 50.1 is an error

Dictionary, set comprehension and generator expression
  dict comprehension {minute: temperature} of alarms: {3: 9.2, 6: 1.5, 10: 8.1, 11: 50.0, 12: -50.0}
  set comprehension of distinct valid whole degrees  : [-50, 1, 2, 3, 4, 5, 7, 8, 9, 50]
  generator expression: generator | first value: 4.5 | second: 3.1
  list(g) now gives only what is left: [9.2, 5.0, 1.5] ... (a generator can be used once)

Edge cases
  all valid    -> valid 5, alarms [], three versions same: True
  all invalid  -> valid 0, alarms [], three versions same: True
  empty        -> valid 0, alarms [], three versions same: True
  1,000,000 all valid  : kept 1000000 | alarms 0
  1,000,000 all invalid: kept 0 | alarms 0

Timings in seconds (best of 3): clean + convert + alarm minutes
  readings     for loop  comprehension   map/filter
     10000      0.00167        0.00150      0.00192   identical results: True
    100000      0.01714        0.01525      0.01953   identical results: True
   1000000      0.19446        0.16289      0.20710   identical results: True

Memory of the valid Celsius readings (sys.getsizeof and tracemalloc peak)
  readings    list comp (B)      generator (B)    peak list sum (B) peak generator sum (B)
     10000            85176                104               85744                  720   sums equal: True
    100000           800984                104              801120                  296   sums equal: True
   1000000          8448728                104             8448864                  296   sums equal: True
```

```
Terminal - python3
   ... (output continues)

Time of sum(list comprehension) vs sum(generator) for 1,000,000 readings
  sum([...]) 0.03957 s | sum(...) generator 0.04232 s
```

5. Design & Analysis

- **General form.** [expression for item in iterable if condition]. For the problem: celsius = [t for t in raw if -50 <= t <= 50] and fahrenheit = [c * 9 / 5 + 32 for c in celsius]. The condition filters, and the expression is what is put in the new list.
- **When it is harder to read.** A comprehension is harder to read than a loop when it has two or more for-parts, more than one or two conditions, an if-else inside the expression, or when it needs a side effect such as printing or updating a counter. For the alarm minutes I kept the condition short and used enumerate; if I also had to log each error, I would go back to the for loop.

- **Dictionary and set comprehension.** Dictionary comprehension: {minute: t for minute, t in enumerate(raw) if -50 <= t <= 50 and not 2 <= t <= 8} gave {3: 9.2, 6: 1.5, 10: 8.1, 11: 50.0, 12: -50.0}. Set comprehension: {int(t) for t in raw if -50 <= t <= 50}, the distinct whole degrees seen.
- **List comprehension against generator expression.** [...] builds the whole list in memory at once. (...) makes a generator that gives one value at a time when asked and keeps nothing. In my test next(g) gave 4.5 and 3.1, and list(g) afterwards gave only the rest, because a generator can be used only once and has no length or index.

6. Experimental Investigation

readings	for loop + append	list comprehensions	map + filter + lambda
10,000	0.00167 s	0.00150 s	0.00192 s
100,000	0.01714 s	0.01525 s	0.01953 s
1,000,000	0.19446 s	0.16289 s	0.20710 s

Table Q5-1: time for clean + convert + alarm minutes, best of 3. The three versions gave identical results at every size.

readings	list comprehension getsizeof (B)	generator getsizeof (B)	peak memory sum([...]) (B)	peak memory sum(generator) (B)
10,000	85,176	104	85,744	720
100,000	800,984	104	801,120	296
1,000,000	8,448,728	104	8,448,864	296

Table Q5-2: sys.getsizeof of the object, and the peak memory seen by tracemalloc while summing.

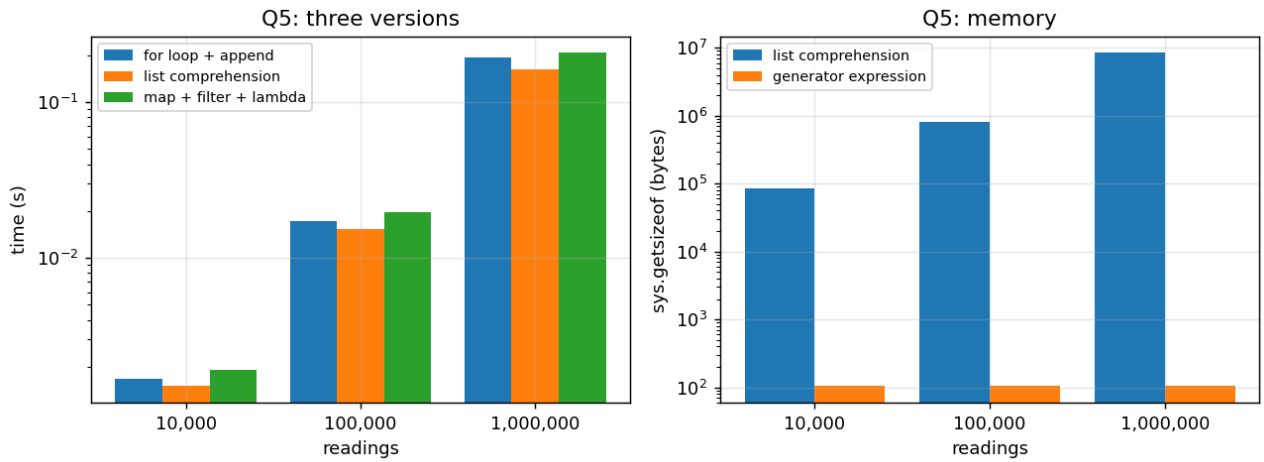


Figure Q5-1: time of the three versions and memory of list comprehension against generator (log scales).

Verification and tests: for the 14-value test list and the 3 big sizes, the three versions gave identical celsius, fahrenheit and alarm lists. All-valid input (1,000,000 readings of 5.0): everything kept, no alarms. All-invalid input (1,000,000 readings of 99.0): nothing kept, no alarms. The comprehension was fastest at 1,000,000 readings (0.163 s), the loop was 1.19 times that and map/filter with lambda 1.27 times, so the prediction held, but the differences are small (10 to 30%). The memory difference is big: a list comprehension of 1,000,000 valid readings was 8,448,728 bytes and a generator was 104 bytes. Summing with tracemalloc showed a peak of 8,448,864 bytes for sum([...]) against 296 bytes for sum(generator).

7. Conclusion

No, a list comprehension is not always the best choice. It was the fastest of the three ways (about 16% faster than the for loop for 1,000,000 readings), and it is short, so it is my first choice when I need the list. But when I only need a sum, a count or one pass, a generator expression does the same job in 104 bytes instead of 8.4 MB, and in my test it was even about the same speed (0.042 s against 0.040 s for the list version). And when the logic needs several steps, a plain for loop is easier to read and debug than a long comprehension.

8. Alternative Concept and Complexity Analysis

Instead of the list comprehension, sum(c for c in raw if -50 <= c <= 50) uses a generator expression: no list is created and the values are added as they are produced. The same idea gives count = sum(1 for t in raw if ...).

Version	Time	Memory	Measured at 1,000,000 readings
sum([c for c in raw if ...])	O(n)	O(n) for the list	0.0396 s, peak 8,448,864 B
sum(c for c in raw if ...)	O(n)	O(1)	0.0423 s, peak 296 B

Table Q5-3: list comprehension against generator expression.

Both visit every reading once, so the time is O(n) in both. The difference is memory: the list needs O(n) (it grows with the input: 85,176 B, 800,984 B and 8,448,728 B in Table Q5-2) while the generator stays O(1) (104 B). The price of the generator is that it can be used only once and cannot be indexed or measured with len().

Q6 - NESTED DATA: A RAILWAY TIMETABLE (7 Marks)

1. Aim

To keep a railway timetable as a dictionary of dictionaries (train number -> name, list of (station, arrival, departure) tuples, set of running days), to find the trains that stop at a station on a day sorted by departure time, and to build and measure a station index for timetables of 100, 1,000 and 10,000 trains.

2. Predictions made before running

- The expression for the departure time of the third station of train 12601 is timetable[12601]["stops"][2][2], which should give 23:02 (Jolarpettai).
- Without an index, one query looks at every stop of every train, so the time grows in proportion to the number of trains (about 10 times for 10 times more trains). With an index it looks only at the trains that stop at that station.
- An unknown station should return an empty list (not an error), and a train that stops at the station but does not run on that day should be left out.
- The index will use only a small part of the memory of the timetable, because it stores only references to train numbers.

3. Program

File: Q6/q6_timetable.py. The 5-train timetable is written by hand with real station names; the big timetables are generated with random.Random(4): 200 stations, 8 stops per train, 2 to 7 running days. The program has trains_at (no index), build_index and trains_at_indexed, and the flat-list alternative (flat_list and trains_at_flat).

```python
"""Q6 - Railway enquiry counter: nested dictionaries, station index, flat-list alternative."""
import json
import random
import sys
import time

DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

# timetable: train number -> {"name": str, "stops": [(station, arrival, departure), ...], "days": {..}}
timetable = {
    12601: {"name": "Chennai Mail", "stops": [("Chennai Central", "20:10", "20:10"), ("Katpadi", "22:05", "22:07"),
                                              ("Jolarpettai", "23:00", "23:02"), ("Bangalore City", "02:30", "02:30")],
            "days": {"Mon", "Wed", "Fri", "Sun"}},
    12007: {"name": "Shatabdi Express", "stops": [("Chennai Central", "06:00", "06:00"), ("Katpadi", "07:40", "07:42"),
                                                  ("Jolarpettai", "08:30", "08:31"), ("Mysuru", "12:30", "12:30")],
            "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat"}},
    22625: {"name": "Double Decker", "stops": [("Chennai Central", "07:25", "07:25"), ("Katpadi", "09:03", "09:05"),
                                              ("Bangalore City", "12:20", "12:20")],
            "days": {"Tue", "Thu", "Sat"}},
    16021: {"name": "Kaveri Express", "stops": [("Mysuru", "14:00", "14:00"), ("Bangalore City", "16:50", "17:00"),
                                               ("Jolarpettai", "19:25", "19:27"), ("Katpadi", "20:20", "20:22"),
                                               ("Chennai Central", "23:30", "23:30")],
            "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"}},
    12657: {"name": "Bangalore Mail", "stops": [("Chennai Central", "22:00", "22:00"), ("Katpadi", "23:52", "23:54"),
                                                ("Bangalore City", "04:00", "04:00")],
            "days": {"Sat", "Sun"}},
}


def trains_at(station, day, tt):
    """no index: look at every stop of every train"""
    found = []
    for number, info in tt.items():
        if day not in info["days"]:
            continue
        for st, arr, dep in info["stops"]:
            if st == station:
                found.append((number, info["name"], arr, dep))
    return sorted(found, key=lambda row: row[3])           # by departure time from the station


def build_index(tt):
    """index: station -> list of train numbers that stop there"""
    index = {}
    for number, info in tt.items():
        for st, arr, dep in info["stops"]:
```

```
            index.setdefault(st, []).append(number)
    return index


def trains_at_indexed(station, day, tt, index):
    found = []
    for number in index.get(station, []):                  # only the trains that stop here
        info = tt[number]
        if day in info["days"]:
            for st, arr, dep in info["stops"]:
                if st == station:
                    found.append((number, info["name"], arr, dep))
    return sorted(found, key=lambda row: row[3])


def flat_list(tt):
    return [(number, st, arr, dep, frozenset(info["days"])) for number, info in tt.items() for st, arr, dep in info["stops"]]


def trains_at_flat(station, day, flat, names):
    found = [(n, names[n], arr, dep) for n, st, arr, dep, days in flat if st == station and day in days]
    return sorted(found, key=lambda row: row[3])


def print_timetable(tt):
    print("%-6s %-17s %s" % ("Train", "Name", "Runs on"))
    for number, info in sorted(tt.items()):
        days = ",".join(d for d in DAYS if d in info["days"]) if len(info["days"]) < 7 else "Daily"
        print("%-6d %-17s %s" % (number, info["name"], days))
        for s, a, d in info["stops"]:
            print("         %-16s arr %s  dep %s" % (s, a, d))


def deep_size(obj, seen):
    if id(obj) in seen:
        return 0
    seen.add(id(obj))
    size = sys.getsizeof(obj)
    if isinstance(obj, dict):
        size += sum(deep_size(k, seen) + deep_size(v, seen) for k, v in obj.items())
    elif isinstance(obj, (list, tuple, set, frozenset)):
        size += sum(deep_size(x, seen) for x in obj)
    return size


def make_timetable(n, seed=4, stations=200, stops=8):
```

```
    rng = random.Random(seed)
    names = ["S%03d" % i for i in range(stations)]
    tt = {}
    for k in range(n):
        route = rng.sample(names, stops)
        h, stop_list = rng.randrange(24), []
        m = rng.randrange(60)
        for st in route:
            arr = "%02d:%02d" % (h % 24, m)
            m += rng.randrange(1, 4)
            h, m = h + m // 60, m % 60
            dep = "%02d:%02d" % (h % 24, m)
            stop_list.append((st, arr, dep))
            m += rng.randrange(20, 50)
            h, m = h + m // 60, m % 60
        tt[10000 + k] = {"name": "Train %d" % k, "stops": stop_list, "days": set(rng.sample(DAYS, rng.randint(2, 7)))}
    return tt, names


results = {}

# ---------- 1. the 5-train timetable ----------
print("Timetable of 5 trains")
print_timetable(timetable)

t12601 = timetable[12601]
print("\nDeparture time of the third station of train 12601: timetable[12601]['stops'][2][2] =", timetable[12601]["stops"][2][2],
      "(station:", timetable[12601]["stops"][2][0] + ")")
print("Types: timetable ->", type(timetable).__name__, "| one train ->", type(t12601).__name__, "| stops ->",
      type(t12601["stops"]).__name__, "| one stop ->", type(t12601["stops"][0]).__name__, "| days ->", type(t12601["days"]).__name__)

# ---------- 2. query ----------
print("\nTrains that stop at Katpadi on Friday, sorted by departure time (sorted + key=lambda):")
index = build_index(timetable)
for number, name, arr, dep in trains_at("Katpadi", "Fri", timetable):
    print("  %d  %-18s arrives %s  departs %s" % (number, name, arr, dep))
print("Same answer with the index:", trains_at("Katpadi", "Fri", timetable) == trains_at_indexed("Katpadi", "Fri", timetable, index))
print("Index (station -> trains):")
for st, trs in index.items():
    print("  %-16s %s" % (st, trs))

print("\nEdge cases")
print("  unknown station 'Delhi' ->", trains_at("Delhi", "Fri", timetable), "| with index:", trains_at_indexed("Delhi", "Fri", timetable, index))
print("  Katpadi on Sunday (12007 and 22625 do not run) ->", [t[0] for t in trains_at("Katpadi", "Sun", timetable)])
print("  22625 stops at Katpadi but does not run on Friday:", 22625 in [t[0] for t in trains_at("Katpadi", "Fri", timetable)])
print("  index.get('Delhi', []) =", index.get("Delhi", []), "| index['Delhi'] would raise KeyError")
```

```
try:
    timetable[99999]
except KeyError as e:
    print("  timetable[99999] raises KeyError:", e)

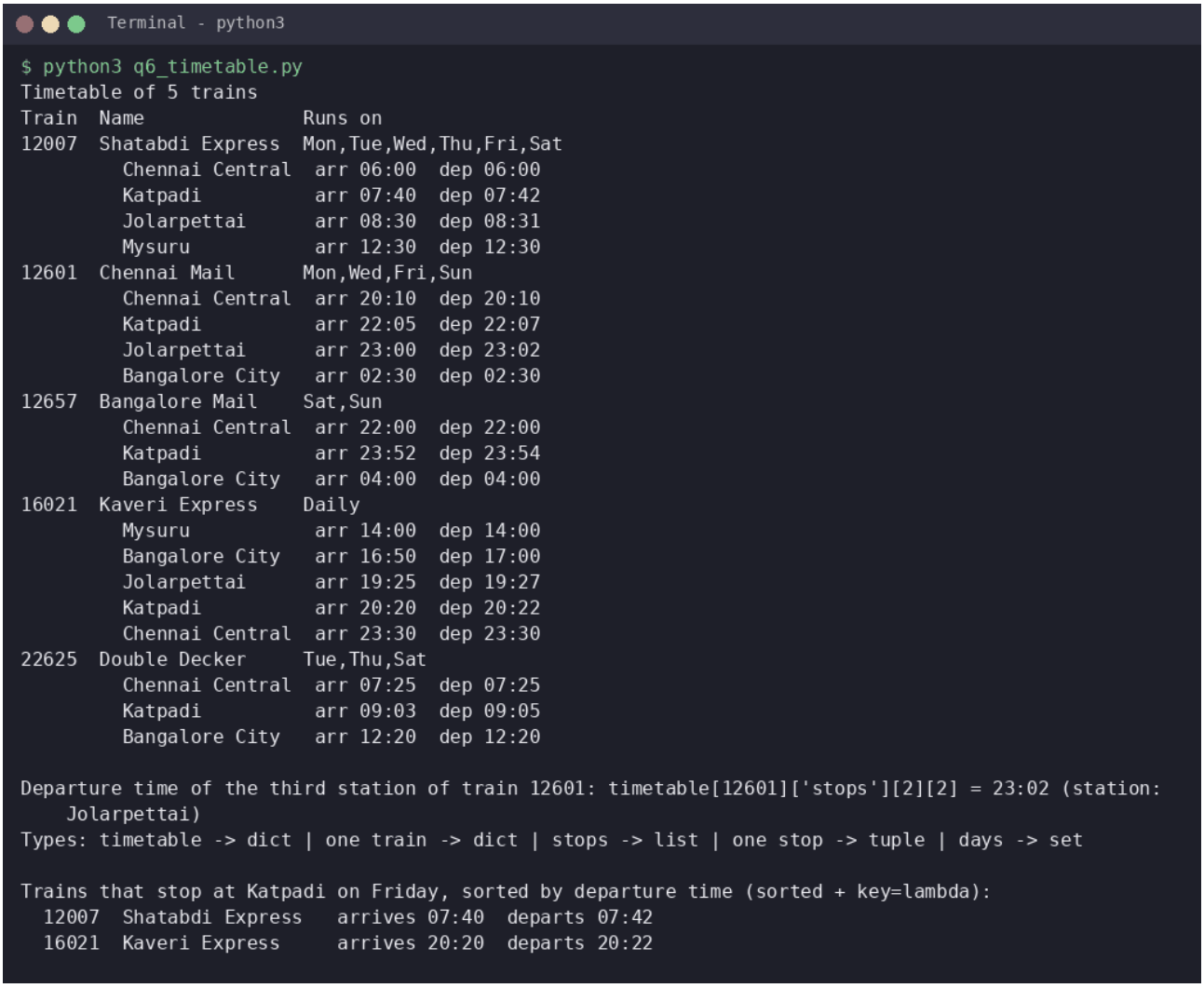
# ---------- 3. big timetables ----------
print("\nStation query: average of 300 queries, microseconds. Each train has 8 stops, 200 stations.")
print("%8s %12s %14s %14s %14s %12s" % ("trains", "total stops", "no index", "with index", "flat list", "index build"))
rows = []
mrows = []
for n in (100, 1000, 10000):
    tt, names = make_timetable(n)
    total = sum(len(i["stops"]) for i in tt.values())
    t0 = time.perf_counter(); idx = build_index(tt); tb = time.perf_counter() - t0
    flat = flat_list(tt)
    nm = {k: v["name"] for k, v in tt.items()}
    rng = random.Random(99)
    qs = [(rng.choice(names), rng.choice(DAYS)) for _ in range(300)]
    t0 = time.perf_counter()
    for s, d in qs:
        r1 = trains_at(s, d, tt)
    t_no = (time.perf_counter() - t0) / 300
    t0 = time.perf_counter()
    for s, d in qs:
        r2 = trains_at_indexed(s, d, tt, idx)
    t_ix = (time.perf_counter() - t0) / 300
    t0 = time.perf_counter()
    for s, d in qs:
        r3 = trains_at_flat(s, d, flat, nm)
    t_fl = (time.perf_counter() - t0) / 300
    same = all(trains_at(s, d, tt) == trains_at_indexed(s, d, tt, idx) == trains_at_flat(s, d, flat, nm) for s, d in qs[:50])
    avg_here = sum(len(v) for v in idx.values()) / len(idx)
    rows.append([n, total, t_no, t_ix, t_fl, tb, avg_here])
    seen = set()
    base = deep_size(tt, seen)             # objects of the timetable are marked as seen
    extra = deep_size(idx, seen)           # only NEW memory used by the index is counted now
    fl_mem = deep_size(flat, set())
    mrows.append([n, base, extra, fl_mem])
    print("%8d %12d %12.1f us %12.1f us %12.1f us %10.4f s   same answers: %s" % (n, total, t_no * 1e6, t_ix * 1e6, t_fl * 1e6, tb, same))
results["times"] = rows
print("\nMemory (sys.getsizeof, every object counted once)")
print("%8s %16s %18s %14s %16s" % ("trains", "timetable (B)", "extra for index (B)", "index / table", "flat list (B)"))
for n, base, extra, fl in mrows:
    print("%8d %16d %18d %13.1f%% %16d" % (n, base, extra, extra / base * 100, fl))
results["mem"] = mrows
```

```
# unknown station / non-running train at scale
tt, names = make_timetable(10000)
idx = build_index(tt)
print("\nAt 10,000 trains: unknown station ->", trains_at_indexed("XYZ", "Mon", tt, idx), "| the busiest station has", max(len(v) for v in idx.values()), "trains, the quietest", min(len(v) for v in idx.values()))

with open("results_q6.json", "w") as f:
    json.dump(results, f)
```

4. Output

Real run (command: python3 q6_timetable.py), Python 3.10.12.


Terminal - python3
$ python3 q6_timetable.py
Timetable of 5 trains
Train Name   Runs on
12007 Shatabdi Express Mon,Tue,Wed,Thu,Fri,Sat
         Chennai Central arr 06:00 dep 06:00
         Katpadi   arr 07:40 dep 07:42
         Jolarpettai   arr 08:30 dep 08:31
         Mysuru   arr 12:30 dep 12:30
12601 Chennai Mail   Mon,Wed,Fri,Sun
         Chennai Central arr 20:10 dep 20:10
         Katpadi   arr 22:05 dep 22:07
         Jolarpettai   arr 23:00 dep 23:02
         Bangalore City   arr 02:30 dep 02:30
12657 Bangalore Mail   Sat,Sun
         Chennai Central arr 22:00 dep 22:00
         Katpadi   arr 23:52 dep 23:54
         Bangalore City   arr 04:00 dep 04:00
16021 Kaveri Express   Daily
         Mysuru   arr 14:00 dep 14:00
         Bangalore City   arr 16:50 dep 17:00
         Jolarpettai   arr 19:25 dep 19:27
         Katpadi   arr 20:20 dep 20:22
         Chennai Central arr 23:30 dep 23:30
22625 Double Decker   Tue,Thu,Sat
         Chennai Central arr 07:25 dep 07:25
         Katpadi   arr 09:03 dep 09:05
         Bangalore City   arr 12:20 dep 12:20
Departure time of the third station of train 12601: timetable[12601]['stops'][2][2] = 23:02 (station:
    Jolarpettai)
Types: timetable -> dict | one train -> dict | stops -> list | one stop -> tuple | days -> set
Trains that stop at Katpadi on Friday, sorted by departure time (sorted + key=lambda):
  12007 Shatabdi Express   arrives 07:40 departs 07:42
  16021 Kaveri Express   arrives 20:20 departs 20:22

```
Terminal - python3

  ... (output continues)
  12601  Chennai Mail       arrives 22:05  departs 22:07
Same answer with the index: True
Index (station -> trains):
  Chennai Central  [12601, 12007, 22625, 16021, 12657]
  Katpadi          [12601, 12007, 22625, 16021, 12657]
  Jolarpettai      [12601, 12007, 16021]
  Bangalore City   [12601, 22625, 16021, 12657]
  Mysuru           [12007, 16021]

Edge cases
  unknown station 'Delhi' -> [] | with index: []
  Katpadi on Sunday (12007 and 22625 do not run) -> [16021, 12601, 12657]
  22625 stops at Katpadi but does not run on Friday: False
  index.get('Delhi', []) = [] | index['Delhi'] would raise KeyError
  timetable[99999] raises KeyError: 99999

Station query: average of 300 queries, microseconds. Each train has 8 stops, 200 stations.
  trains  total stops       no index     with index      flat list  index build
     100          800        16.8 us         1.3 us         19.5 us     0.0001 s   same answers: True
    1000         8000       204.0 us        12.8 us        219.4 us     0.0007 s   same answers: True
   10000        80000      2413.1 us       243.0 us       2599.5 us     0.0063 s   same answers: True

Memory (sys.getsizeof, every object counted once)
  trains    timetable (B) extra for index (B)  index / table    flat list (B)
     100           244451              29048          11.9%           447977
    1000          2336534              92832           4.0%          4372428
   10000         23181670             702752           3.0%         43589548

At 10,000 trains: unknown station -> [] | the busiest station has 456 trains, the quietest 345
```

5. Design & Analysis

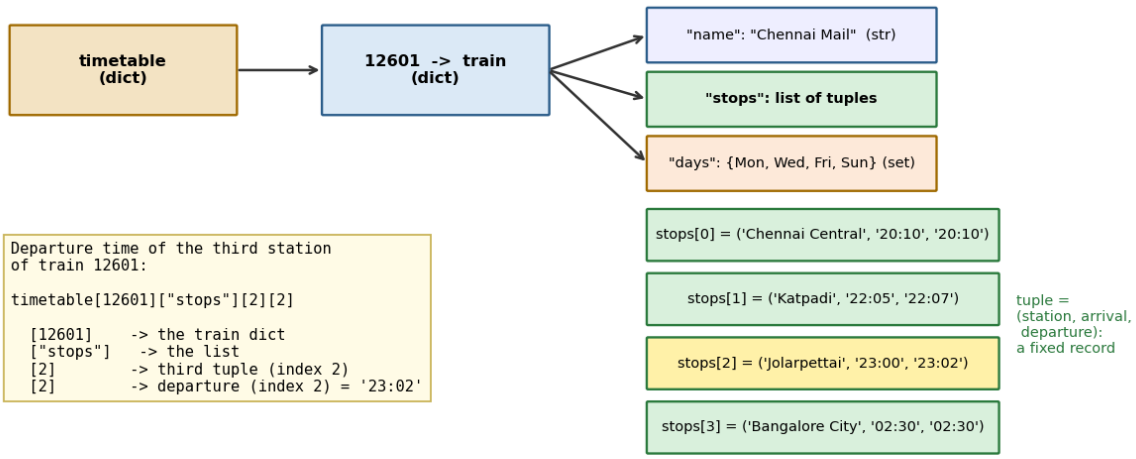


Figure Q6-1: the nested structure for train 12601 and the path to the departure time of its third station.

- **Expression.** timetable[12601]["stops"][2][2] gave 23:02, the departure from Jolarpettai, as predicted. [12601] selects the train dictionary, ["stops"] the list, [2] the third tuple and the last [2] the departure inside the tuple (station is 0, arrival 1, departure 2).
- **Why dictionary, list, tuple and set.** The outer and the train level are dictionaries because I look things up by a key (train number, "name", "stops", "days"). The stops are a list because their order matters (the route) and I may need to go through them in order. Each stop is a tuple (station, arrival, departure) because it is a fixed record that should not be changed by accident. The running days are a set because only membership matters ("does it run on Fri?") and a day cannot appear twice, so day in info["days"] is O(1).
- **Work for one query.** Without an index: for each of T trains, check the day (O(1)), then scan its s stops for the station: O(T x s), which is the total number of stops, plus sorting the k trains found, O(k log k). With an index (station -> list of trains): one dictionary lookup O(1), then only the k trains at that station, each scanned for its s stops, so O(k x s + k log k), and k is much smaller than T.
- **Index.** The index is built once: for every stop of every train, index.setdefault(station, []).append(train_number). Building is O(total stops) (0.0064 s for 80,000 stops) and must be repeated (or updated) when the timetable changes.

6. Experimental Investigation

Correctness tests: for the 5-train timetable, Katpadi on Friday gave 12007 (07:42), 16021 (20:22) and 12601 (22:07) in order of departure, and the indexed version gave the same list. An unknown station (Delhi) gave [] with and without the index. Katpadi on Sunday left out 12007 and 22625 because they do not run on Sunday, and on Friday 22625 was left out for the same reason although it stops at Katpadi. index.get("Delhi", []) is safe, while index["Delhi"] and timetable[99999] raise KeyError.

trains	total stops	query without index	query with index	query on flat list	speed-up of index	index build time
100	800	16.8 us	1.3 us	19.5 us	12.5x	0.0001 s
1,000	8,000	204.0 us	12.8 us	219.4 us	15.9x	0.0007 s
10,000	80,000	2413.1 us	243.0 us	2599.5 us	9.9x	0.0063 s

Table Q6-1: average of 300 random (station, day) queries; the three methods gave the same answers (checked on 50 queries per size).

trains	timetable (B)	extra memory of the index (B)	index / timetable	flat list (B)
100	244,451	29,048	11.9%	447,977
1,000	2,336,534	92,832	4.0%	4,372,428
10,000	23,181,670	702,752	3.0%	43,589,548

Table Q6-2: memory with sys.getsizeof, each object counted once. The index figure counts only the new dictionary and lists (train numbers are shared with the timetable). The flat list is counted on its own, so it overstates what it would add.

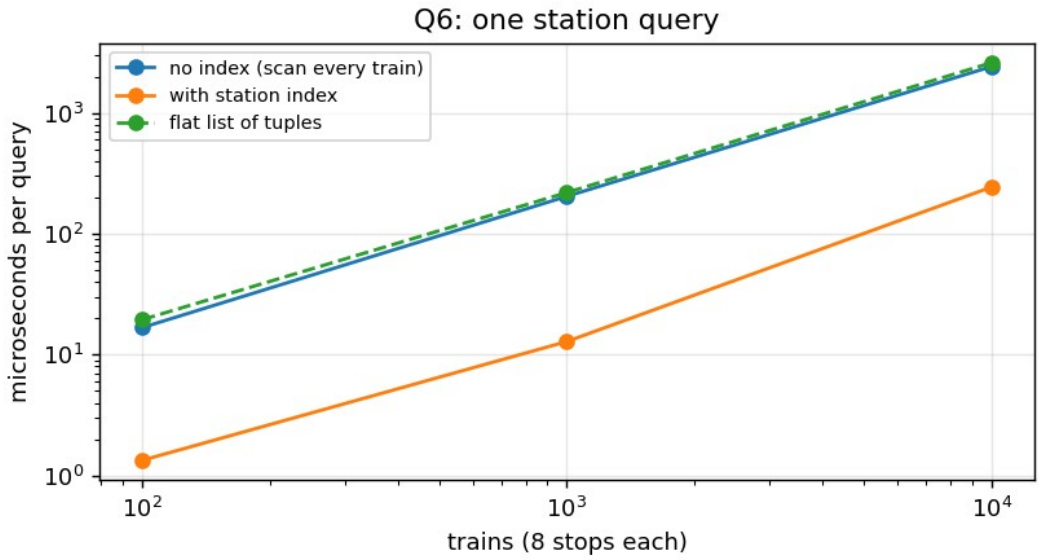


Figure Q6-2: time for one station query against the number of trains (log scales).

The time without an index grew 10 times for each 10 times more trains (16.8 us, 204.0 us, 2413.1 us), as predicted for O(total stops). The indexed query grew in the same way here, because in my random timetable each station is used by about 4% of the trains (345 to 456 of 10,000), so k also grows with T. The index was about 10 times faster at 10,000 trains. The paper estimate is about 22 times (80,000 stops against about 3,600 stops in the 450 trains at a station); I did not find out why I got 10 and not 22, my guess is that the sort and building the result list cost the same in both methods. The extra memory of the index was only 3.0% of the timetable at 10,000 trains (702,752 bytes against 23,181,670).

7. Conclusion

An index makes each query fast because the work is done once, when the index is built: it already knows which trains stop at each station, so a query does not have to look at the other trains. The trade-off is memory (about 3% extra here), the time to build it, and keeping it correct: if a train is added or its stops change, the index must be updated too, otherwise a query will give an old answer. It is worth it when there are many queries for each change, as at an enquiry counter.

8. Alternative Concept and Complexity Analysis

The alternative is one flat list of tuples (train, station, arrival, departure, days), scanned in full for every query: [row for row in flat if row[1] == station and day in row[4]], then sorted by departure. There is no nesting and no index, which is simple, but each query has to look at every stop.

Design	Time per query	Space	Measured at 10,000 trains
nested dictionaries, no index	O(total stops)	O(total stops)	2413.1 us
nested dictionaries + station index	O(trains at the station x stops) + O(1) lookup	O(total stops) + O(total stops) references for the index	243.0 us
flat list of tuples	O(total stops)	O(total stops), one tuple per stop	2599.5 us

Table Q6-3: complexity and measurement.

The flat list took 1.08 times as long as the nested dictionaries without the index, so it was no better in time: both scan all 80,000 stops. The index is the only one of the three that avoids scanning everything. The flat list repeats the train number and the days in every row, so it needs more space than the nested structure, which stores them once per train; that is the strongest reason to prefer the nested dictionaries.

Q7 - DESIGN A CONTACT-MERGING PROGRAM FROM FIRST PRINCIPLES (10 Marks)

I started from the requirements only and then designed, implemented, measured and defended it, in the four stages of the question.

Stage 1: Interpret the Problem (2 marks)

Input: a list of contacts coming from two old phones and a SIM card. Each contact has a name (any spelling), zero or more phone numbers (any format) and zero or more e-mail addresses. Output: a list of people, each person once, with all their numbers (without repeats) and all their e-mail addresses.

- **Rule for "same person".** Two contacts are the same person if they share at least one normalised phone number, or (when a contact has no usable number) at least one e-mail address. This is also transitive: if A shares a number with B and B shares a number with C, then A, B and C are one person.
- **What is not a rule.** The name is NOT used to decide, because "Dr. Ravi" and "ravi kumar" are the same person but two different people can also be called "Ravi Kumar". The names are only collected and used to show a name.
- **Data structures.** A dictionary keyed by the normalised number (finds the person for a number in O(1)); a set of numbers and a set of e-mails per person (no repeats, so the same number written twice is stored once); a set of the names seen; and a list of the people records.

#	Measurable requirement	How I check it
R1	"+91 98400 12345", "098400-12345", "9840012345" and 5 other formats and the other formats in the test all normalise to the same 10-digit string	normalisation test table
R2	Contacts with a common number or e-mail end up in one person (0 people wrongly split)	generated data with known truth, 1,000 and 100,000 contacts
R3	Contacts that share nothing (including two people with the same name) stay separate	hand-made small list
R4	No number and no e-mail is lost or invented (set of inputs = set of outputs)	check in the program
R5	Merging 100,000 contacts takes less than 2 seconds on my computer, and 1,000 contacts less than 50 ms	timing, best of 3
R6	The merge is O(n): time per contact about the same for 1,000 and 100,000	compare us per contact

Table Q7-1: requirements stated as measurable statements.

Stage 2: Design (3 marks)

Normalisation. Phone: remove everything that is not a digit; if what remains is 10 digits plus a country or trunk prefix (0091, 091, 91 or 0), remove the prefix; accept only exactly 10 digits, otherwise reject the number (it is reported, not silently lost). E-mail: strip spaces, lower-case, accept only if it looks like x@y.z. Name: lower-case, replace punctuation with spaces, split into words and drop titles (dr, mr, mrs, ms, prof, sri, shri, smt, er).

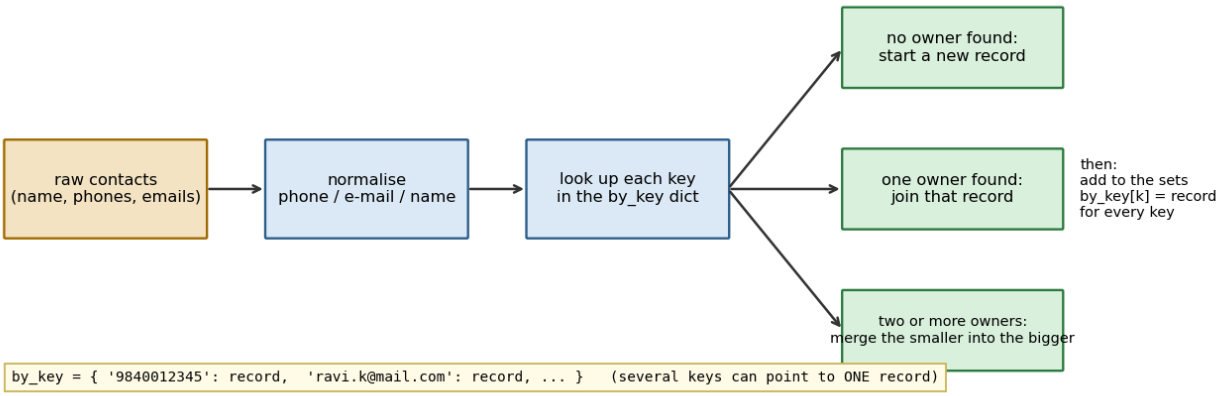


Figure Q7-1: design of the merge. Every normalised number (and every e-mail of a contact with no number) points to ONE shared record.

Pseudocode of the merge:

```
by_key = empty dictionary     # key (number or e-mail) -> person record
for each contact c:
    nums  = {normalise_phone(p) for p in c.phones} without rejected
    mails = {normalise_email(e) for e in c.emails} without rejected
    keys  = nums if nums is not empty else mails
    owners = different records found by looking up each key
    if owners is empty:
        person = new record (empty sets: names, numbers, emails)
    else:
        person = the biggest of owners
        merge every other owner into person (union the sets)
        point the keys of those owners to person
    person.names.add(normalise_name(c.name))
    person.numbers |= nums ; person.emails |= mails
    for every key in nums and mails: by_key[key] = person
result = records not merged into another one, sorted by name
```

Why sets and a dictionary: the set removes repeated numbers and e-mails automatically; the dictionary makes "who owns this number?" a single O(1) lookup, so each contact costs a constant amount of work and the whole merge is O(n) on average. The "merge the smaller into the bigger" step is needed when a contact links two groups that were separate until now (for example A has number 1, B has number 2, and a later contact has both numbers).

Stage 3: Implement and Measure (3 marks)

File: Q7/q7_contacts.py (normalisation, merge, the pair-comparison alternative, a data generator with known truth, and the measurements).

```python
"""Q7 - Merge phone contacts: normalise, group with a dictionary keyed by normalised number."""
import json
import random
import re
import sys
import time

TITLES = {"dr", "mr", "mrs", "ms", "prof", "sri", "shri", "smt", "er"}


# ---------------- Stage 2: normalisation ----------------
def norm_phone(raw):
    """'+91 98400 12345', '098400-12345', '9840012345' -> '9840012345'. Returns None if not usable."""
    digits = re.sub(r"\D", "", raw or "")
    for prefix in ("0091", "091", "91", "0"):
        if digits.startswith(prefix) and len(digits) - len(prefix) == 10:
            digits = digits[len(prefix):]
            break
    return digits if len(digits) == 10 else None


def norm_email(raw):
    raw = (raw or "").strip().lower()
    return raw if re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", raw) else None


def norm_name(raw):
    """'Dr. Ravi' -> 'ravi', ' RAVI   Kumar ' -> 'ravi kumar'"""
    words = re.sub(r"[^\w\s]", " ", (raw or "").lower()).split()
    return " ".join(w for w in words if w not in TITLES)


# ---------------- Stage 2: merging ----------------
def merge(contacts):
    """contacts: list of dicts {name, phones: [..], emails: [..]}.
    by_key maps every normalised number (and every e-mail when the contact has no number) to ONE shared record."""
    by_key = {}
    people = []                  # all records ever created
    rejected = []                # raw numbers that could not be normalised
    for c in contacts:
        nums = set()
        for p in c.get("phones", []):
            n = norm_phone(p)
            if n:
                nums.add(n)
            elif p:
```

```
                rejected.append(p)
        mails = {m for m in (norm_email(e) for e in c.get("emails", [])) if m}
        name = norm_name(c.get("name", ""))
        keys = list(nums) if nums else list(mails)
        # find the records that already own one of my keys
        owners = []
        for k in keys:
            rec = by_key.get(k)
            if rec is not None and all(rec is not o for o in owners):
                owners.append(rec)
        if not owners:
            rec = {"names": set(), "numbers": set(), "emails": set(), "alive": True}
            people.append(rec)
        else:
            rec = max(owners, key=lambda r: len(r["numbers"]) + len(r["emails"]))
            for other in owners:                    # two groups joined by this contact: merge the smaller ones in
                if other is not rec:
                    rec["names"] |= other["names"]; rec["numbers"] |= other["numbers"]; rec["emails"] |= other["emails"]
                    other["alive"] = False
                    for k in other["numbers"] | other["emails"]:
                        by_key[k] = rec
        if name:
            rec["names"].add(name)
        rec["numbers"] |= nums
        rec["emails"] |= mails
        for k in nums | mails:
            by_key[k] = rec
    merged = [r for r in people if r["alive"]]
    merged.sort(key=lambda r: (display_name(r), sorted(r["numbers"])))
    return merged, rejected


def display_name(rec):
    return max(rec["names"], key=lambda n: (len(n), n)).title() if rec["names"] else "(no name)"


# ---------------- the slow alternative: compare every pair ----------------
def merge_pairs(contacts):
    n = len(contacts)
    sets = []
    for c in contacts:
        s = {norm_phone(p) for p in c.get("phones", [])} - {None}
        s |= {norm_email(e) for e in c.get("emails", [])} - {None}
        sets.append(s)
    parent = list(range(n))
```

```
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for i in range(n):
        for j in range(i + 1, n):
            if not sets[i].isdisjoint(sets[j]):
                parent[find(i)] = find(j)
    groups = {}
    for i in range(n):
        groups.setdefault(find(i), set()).update(sets[i])
    return sorted(sorted(g) for g in groups.values())


# ---------------- test data ----------------
def fmt_phone(rng, n):
    a, b = n[:5], n[5:]
    return rng.choice(["+91 %s %s" % (a, b), "+91%s" % n, n, "0%s" % n, "%s-%s" % (a, b), "91-%s" % n, "(+91) %s" % n, "00 91 %s" % n])


def make_contacts(n, seed=5):
    """about n/2.2 people; every contact lists the person's main number (in some format) + optional extras"""
    rng = random.Random(seed)
    people = max(1, int(n / 2.2))
    first = ["ravi", "priya", "karthik", "anitha", "suresh", "divya", "arun", "meena", "vijay", "lakshmi"]
    last = ["kumar", "raj", "s", "devi", "mohan", "babu", "nair", "iyer"]
    truth = []
    for i in range(people):
        main = "9%09d" % ((i * 7919 + 13) % 10 ** 9)        # distinct for every person (7919 and 10**9 have no common factor)
        extra = "8%09d" % ((i * 104729 + 5) % 10 ** 9)
        truth.append((rng.choice(first) + " " + rng.choice(last) + str(i), main, extra, "p%d@mail.com" % i))
    contacts = []
    owner_of = []
    for k in range(n):
        i = k % people if k < people else rng.randrange(people)
        nm, main, extra, mail = truth[i]
        parts = nm.split()
        spelled = rng.choice([nm, nm.upper(), "Dr. " + parts[0].title(), parts[0], "  " + nm.title() + " "])
        phones = [fmt_phone(rng, main)] + ([fmt_phone(rng, extra)] if rng.random() < 0.3 else [])
        emails = [mail.upper() if rng.random() < 0.5 else mail] if rng.random() < 0.4 else []
        contacts.append({"name": spelled, "phones": phones, "emails": emails})
        owner_of.append(i)
    return contacts, owner_of, people
```

```python
def best(fn, *a, repeat=3):
    b = 1e9
    for _ in range(repeat):
        t = time.perf_counter(); fn(*a); b = min(b, time.perf_counter() - t)
    return b


results = {}

# ---------- Stage 3a: normalisation tests ----------
print("Phone normalisation (input -> output)")
for raw in ["+91 98400 12345", "9840012345", "098400-12345", "91-9840012345", "(+91) 98400 12345", "00 91 98400 12345", "98400 1234", "044-2345 6789", "", "abc"]:
    print("  %-22r -> %r" % (raw, norm_phone(raw)))
print("\nName normalisation")
for raw in ["Dr. Ravi", "ravi kumar", "  RAVI   KUMAR ", "Mrs. Priya S.", ""]:
    print("  %-18r -> %r" % (raw, norm_name(raw)))
print("\nE-mail normalisation")
for raw in ["Ravi.K@Mail.COM ", "not-an-email", None]:
    print("  %-20r -> %r" % (raw, norm_email(raw)))

# ---------- Stage 3b: a small list merged by hand ----------
small = [
    {"name": "Dr. Ravi", "phones": ["+91 98400 12345"], "emails": []},                          # 1
    {"name": "ravi kumar", "phones": ["9840012345", "044-2345 6789"], "emails": ["Ravi.K@mail.com"]},   # 2 same number as 1 (+ a Chennai landline)
    {"name": "RAVI KUMAR", "phones": [], "emails": ["ravi.k@MAIL.com"]},                        # 3 no number, same e-mail as 2
    {"name": "Priya S", "phones": ["098765-43210"], "emails": []},                              # 4
    {"name": "Priya Sharma", "phones": ["+91 98765 43210", "91-9000011111"], "emails": []},     # 5 same person as 4, new number
    {"name": "Ravi Kumar", "phones": ["+91 90000 22222"], "emails": []},                        # 6 SAME NAME as 2 but a different person (different number)
    {"name": "Anil", "phones": ["98400 1234"], "emails": []},                                   # 7 number too short: rejected, nothing to match on
    {"name": "Dr. Ravi", "phones": ["+91 77777 88888"], "emails": []},                          # 8 same name as 1, different person
    {"name": "Meena", "phones": ["99999 00000", "88888 00000"], "emails": []},                  # 9
    {"name": "M. Meena", "phones": ["88888 00000"], "emails": []},                              # 10 joins 9 by the second number
    {"name": "Meena R", "phones": ["99999-00000"], "emails": ["meena@mail.com"]},               # 11
]
print("\nSmall test: %d contacts" % len(small))
for i, c in enumerate(small, 1):
    print("  %2d %-14s %-38s %s" % (i, c["name"], c["phones"], c["emails"]))
merged, rejected = merge(small)
print("\nBy hand I expect: {1,2,3} one Ravi (same number, then same e-mail) | {4,5} one Priya (same number) | {6} alone (same name as 2, different number) | {7} alone, its number is too short | {8} alone (same name as 1, different number) | {9,10,11} one Meena (10 joins 9 by the second number, 11 joins by the first)")
print("That is 6 people from 11 contacts, and the one rejected number is '98400 1234'.")
print("\nProgram output: %d people" % len(merged))
for r in merged:
    print("  %-14s numbers=%s emails=%s names=%s" % (display_name(r), sorted(r["numbers"]), sorted(r["emails"]), sorted(r["names"])))
print("rejected numbers:", rejected)
nums_in = {norm_phone(p) for c in small for p in c["phones"]} - {None}
```

```python
nums_out = set().union(*(r["numbers"] for r in merged))
mails_in = {norm_email(e) for c in small for e in c["emails"]} - {None}
mails_out = set().union(*(r["emails"] for r in merged))
print("no number lost:", nums_in == nums_out, "| no e-mail lost:", mails_in == mails_out)
print("pair-comparison version finds the same groups:", sorted(sorted(r["numbers"] | r["emails"]) for r in merged if r["numbers"] | r["emails"]) == sorted(g for g in merge_pairs(small) if g))

# ---------- Stage 4 evidence: the limitation ----------
family = [{"name": "Appa", "phones": ["044 2345 6789"]}, {"name": "Amma", "phones": ["+91 44 2345 6789"]}]
fm, _ = merge(family)
print("\nLimitation test: two different people who share one family phone ->", len(fm), "record with names", sorted(fm[0]["names"]))

# ---------- Stage 3c: edge cases ----------
print("\nEdge cases")
print("  empty list ->", merge([]))
print("  contact with no fields at all ->", len(merge([{}])[0]), "record")
print("  same contact repeated 3 times ->", len(merge([small[0]] * 3)[0]), "record")
print("  all different people ->", len(merge([{"name": "A", "phones": ["9%09d" % i]} for i in range(50)])[0]), "records for 50 contacts")

# ---------- Stage 3d: timings ----------
print("\nTiming (best of 3), dictionary merge")
print("%9s %10s %12s %12s %14s" % ("contacts", "people", "groups found", "time (s)", "us per contact"))
rows = []
for n in (1000, 100000):
    contacts, owner_of, people = make_contacts(n)
    t = best(merge, contacts)
    m, rej = merge(contacts)
    # check against the truth: contacts that came from the same person must be in the same group
    wrong = 0
    first_group = {}
    for rec in m:
        pass
    key_of = {}
    for g, rec in enumerate(m):
        for num in rec["numbers"]:
            key_of[num] = g
    group_of_person = {}
    for c, p in zip(contacts, owner_of):
        g = key_of[norm_phone(c["phones"][0])]
        if group_of_person.setdefault(p, g) != g:
            wrong += 1
    present = len({p for p in owner_of})
    rows.append([n, present, len(m), t, wrong])
    print("%9d %10d %12d %12.4f %14.2f   wrongly split: %d" % (n, present, len(m), t, t / n * 1e6, wrong))
results["dict"] = rows
print("  (people = people I put in the test data; wrongly split = people whose contacts ended in more than one group)")
```

```python
# ---------- pair comparison for the alternative ----------
print("\nPair comparison (every contact against every other) - same data, same answers")
print("%9s %12s %14s %12s %10s" % ("contacts", "pairs", "time (s)", "dict (s)", "same groups"))
prow = []
for n in (500, 1000, 2000, 4000):
    contacts, owner_of, people = make_contacts(n)
    tp = best(merge_pairs, contacts, repeat=1)
    td = best(merge, contacts)
    a = merge_pairs(contacts)
    b = sorted(sorted(r["numbers"] | r["emails"]) for r in merge(contacts)[0])
    prow.append([n, n * (n - 1) // 2, tp, td])
    print("%9d %12d %14.4f %12.5f %10s" % (n, n * (n - 1) // 2, tp, td, a == b))
results["pairs"] = prow
# estimate for 100,000 by n^2 scaling from the 4,000 run
n0, _, t0, _ = prow[-1]
est = t0 * (100000 / n0) ** 2
print("Estimate for 100,000 contacts: %.0f pairs x (%.2e s per pair) = %.0f s (%.1f hours)" % (100000 * 99999 / 2, t0 / (n0 * (n0 - 1) / 2), est, est / 3600))
results["estimate_100k_s"] = est

# ---------- memory ----------
print("\nMemory")
for n in (1000, 100000):
    contacts, _, _ = make_contacts(n)
    m, _ = merge(contacts)
    by = sum(sys.getsizeof(r["numbers"]) + sys.getsizeof(r["emails"]) + sys.getsizeof(r["names"]) + sys.getsizeof(r) for r in m)
    print("  n=%7d: %d records, sets+record dicts = %d B (%.0f B per merged person)" % (n, len(m), by, by / len(m)))
    results.setdefault("mem", []).append([n, len(m), by])

with open("results_q7.json", "w") as f:
    json.dump(results, f)
```

Real run (command: python3 q7_contacts.py), Python 3.10.12.

```
Terminal - python3
```

```
$ python3 q7_contacts.py
Phone normalisation (input -> output)
  '+91 98400 12345'      -> '9840012345'
  '9840012345'           -> '9840012345'
  '098400-12345'         -> '9840012345'
  '91-9840012345'        -> '9840012345'
  '(+91) 98400 12345'    -> '9840012345'
  '00 91 98400 12345'    -> '9840012345'
  '98400 1234'           -> None
  '044-2345 6789'        -> '4423456789'
  ''                     -> None
  'abc'                  -> None

Name normalisation
  'Dr. Ravi'         -> 'ravi'
  'ravi kumar'       -> 'ravi kumar'
  '  RAVI   KUMAR '  -> 'ravi kumar'
  'Mrs. Priya S.'    -> 'priya s'
  ''                 -> ''

E-mail normalisation
  'Ravi.K@Mail.COM '   -> 'ravi.k@mail.com'
  'not-an-email'       -> None
  None                 -> None

Small test: 11 contacts
   1 Dr. Ravi       ['+91 98400 12345']                    []
   2 ravi kumar     ['9840012345', '044-2345 6789']        ['Ravi.K@mail.com']
   3 RAVI KUMAR     []                                     ['ravi.k@MAIL.com']
   4 Priya S        ['098765-43210']                       []
   5 Priya Sharma   ['+91 98765 43210', '91-9000011111']   []
   6 Ravi Kumar     ['+91 90000 22222']                    []
   7 Anil           ['98400 1234']                         []
   8 Dr. Ravi       ['+91 77777 88888']                    []
   9 Meena          ['99999 00000', '88888 00000']         []
```

```
Terminal - python3
  ... (output continues)
  10 M. Meena       ['88888 00000']                      []
  11 Meena R        ['99999-00000']                      ['meena@mail.com']

By hand I expect: {1,2,3} one Ravi (same number, then same e-mail) | {4,5} one Priya (same number) | {6} alone
    (same name as 2, different number) | {7} alone, its number is too short | {8} alone (same name as 1,
    different number) | {9,10,11} one Meena (10 joins 9 by the second number, 11 joins by the first)
That is 6 people from 11 contacts, and the one rejected number is '98400 1234'.

Program output: 6 people
  Anil           numbers=[] emails=[] names=['anil']
  Meena R        numbers=['8888800000', '9999900000'] emails=['meena@mail.com'] names=['m meena', 'meena',
    'meena r']
  Priya Sharma   numbers=['9000011111', '9876543210'] emails=[] names=['priya s', 'priya sharma']
  Ravi           numbers=['7777788888'] emails=[] names=['ravi']
  Ravi Kumar     numbers=['4423456789', '9840012345'] emails=['ravi.k@mail.com'] names=['ravi', 'ravi kumar']
  Ravi Kumar     numbers=['9000022222'] emails=[] names=['ravi kumar']
rejected numbers: ['98400 1234']
no number lost: True | no e-mail lost: True
pair-comparison version finds the same groups: True

Limitation test: two different people who share one family phone -> 1 record with names ['amma', 'appa']

Edge cases
  empty list -> ([], [])
  contact with no fields at all -> 1 record
  same contact repeated 3 times -> 1 record
  all different people -> 50 records for 50 contacts

Timing (best of 3), dictionary merge
 contacts     people groups found     time (s) us per contact
     1000        454          454       0.0044          4.38   wrongly split: 0
   100000      45454        45454       0.6159          6.16   wrongly split: 0
  (people = people I put in the test data; wrongly split = people whose contacts ended in more than one group)
```

```
Terminal - python3
  ... (output continues)
Pair comparison (every contact against every other) - same data, same answers
 contacts        pairs       time (s)     dict (s) same groups
      500       124750         0.0092      0.00227       True
     1000       499500         0.0357      0.00462       True
     2000      1999000         0.1302      0.00961       True
     4000      7998000         0.5044      0.01964       True
Estimate for 100,000 contacts: 4999950000 pairs x (6.31e-08 s per pair) = 315 s (0.1 hours)

Memory
  n=   1000: 454 records, sets+record dicts = 399520 B (880 B per merged person)
  n= 100000: 45454 records, sets+record dicts = 39999520 B (880 B per merged person)
```

Test with duplicates, missing fields and different formats (small list of 11 contacts). I worked out the result by hand before running the program: {1, 2, 3} one Ravi (1 and 2 share the number 98400 12345, and 3 has the e-mail of 2), {4, 5} one Priya, {6} alone, {7} alone (its number 98400 1234 has only 9 digits and is rejected), {8} alone, and {9, 10, 11} one Meena (10 joins 9 by the second number and 11 joins by the first); that is 6 people from 11 contacts. The program printed exactly these 6 people. All numbers and e-mails were kept (R4), the one rejected number was reported, and the pair-comparison version found the same groups. The same name in contacts 2 and 6 ("Ravi Kumar") and in contacts 1 and 8 ("Dr. Ravi") did not

cause a wrong merge (R3). Empty list, a contact with no fields, one contact repeated 3 times and 50 different people also behaved correctly.

contacts	people in my test data	groups found	wrongly split people	time	time per contact
1,000	454	454	0	0.0044 s	4.38 us
100,000	45,454	45,454	0	0.6159 s	6.16 us

Table Q7-2: dictionary merge, best of 3 (data generated with every person having a main number written in 8 different formats, extra numbers, e-mails, 5 spellings of the name).

R2: 0 people wrongly split and the number of groups equals the number of people at both sizes. R5: 1,000 contacts took 0.0044 s (limit 50 ms) and 100,000 contacts 0.6159 s (limit 2 s). R6: 4.38 us per contact at 1,000 and 6.16 us at 100,000 contacts, so a 100 times bigger input took about 141 times longer, not 10,000 times; the small extra comes from bigger dictionaries and memory effects, so it is close to linear. Memory: about 880 bytes per merged person (sets and record dictionaries only, 39,999,520 bytes for 45,454 people).

Stage 4: Defend the Design (2 marks)

- **Dictionary against comparing every pair.** Comparing every pair needs n(n - 1)/2 comparisons: 499,500 for 1,000 contacts and about 5 billion for 100,000. With a dictionary each contact does a few lookups, so the cost is about n. I measured it on the same data (Table Q7-3): for 4,000 contacts the pair method took 0.5044 s against 0.0196 s for the dictionary, and the dictionary result was the same in every test. From the n-squared growth, the pair method would need about 315 seconds (5 minutes) for 100,000 contacts (estimate), against 0.6159 s.
- **Two different people with the same name.** The name is not the key. Two different people called "Ravi Kumar" with different numbers get different records (contacts 2 and 6 in my test), and one person written as "Dr. Ravi", "ravi kumar" and "RAVI KUMAR" is joined only because of the shared number or e-mail. The names are only stored in a set and the longest one is shown.
- **Remaining limitation.** Two different people who share a number are merged by mistake. My test: "Appa" and "Amma" with the same family landline became 1 record with the names ['amma', 'appa']. The design trusts that a number belongs to one person. A second limitation of the same kind: a contact with only a name (no number, no e-mail) can never be merged with anything, so "Anil" stays a separate record even if another "Anil" with a number exists. A way to reduce the first problem would be to flag numbers that are used by many different names and ask the user, but I did not implement that.

Alternative Concept and Complexity Analysis

Alternative: compare every pair of contacts, and join two contacts when their sets of numbers or e-mails have something in common (merge_pairs, with a small union-find to make the groups transitive).

contacts	pairs n(n-1)/2	compare every pair	dictionary	same groups?
500	124,750	0.0092 s	0.00227 s	yes
1,000	499,500	0.0357 s	0.00462 s	yes
2,000	1,999,000	0.1302 s	0.00961 s	yes
4,000	7,998,000	0.5044 s	0.01964 s	yes
100,000	4,999,950,000	315 s (estimate)	0.6159 s	-

Table Q7-3: measured at 500 to 4,000 contacts on the same data; the 100,000 row is the pair time extended by n squared, not a measurement.

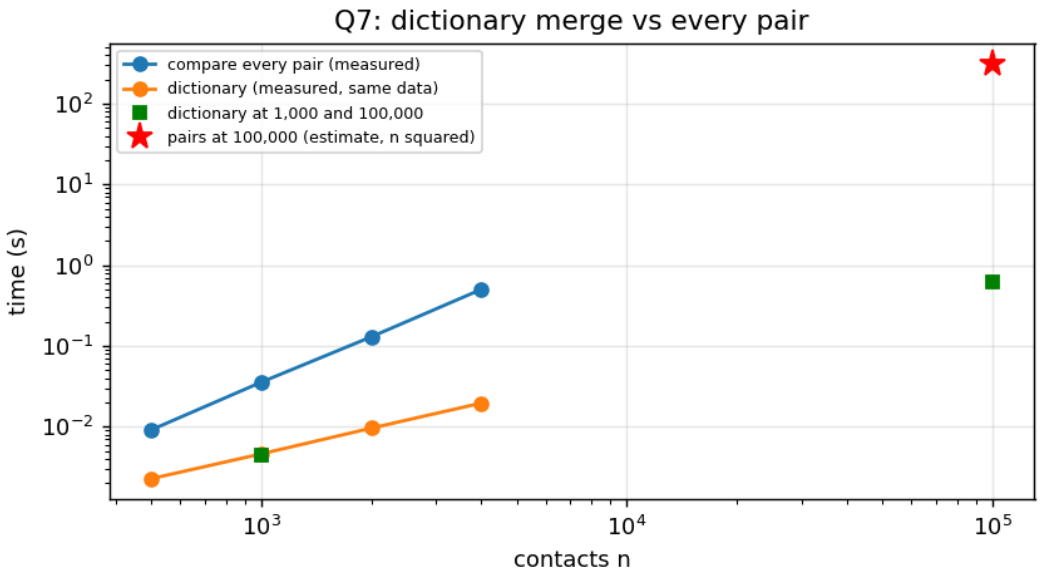


Figure Q7-2: time against the number of contacts (log scales).

Time: comparing every pair is O(n squared) (the time grew about 56 times for 8 times more contacts, between 500 and 4,000: close to the 64 predicted by n squared); the dictionary is O(n) on average (about 9 times for 8 times more). Space: the pair method needs only the set of keys of each contact, O(n) in total and no extra index, the dictionary method needs O(n) for the dictionary and the records (about 880 bytes per person here). So the dictionary uses more memory by a constant factor but changes the time from n squared to n, which is what makes 100,000 contacts possible.

Conclusion

Starting from the requirements gave the design: the rule "same number means same person" decided that the key must be the normalised number, and the problem "one contact can join two groups" decided that several keys must point to the same shared record. The requirements R1 to R6 were all met in my tests; the main weakness is that a shared number merges two people, and I would need to ask the user in that case.